Roma Tre

INGEGNERIA
CIVILE, INFORMATICA E DELLE TECNOLOGIE AERONAUTICHE

Information Visualization

Orthogonal Drawings via Network Flows

M. Patrignani

 copyright ©2025 m.patrignani

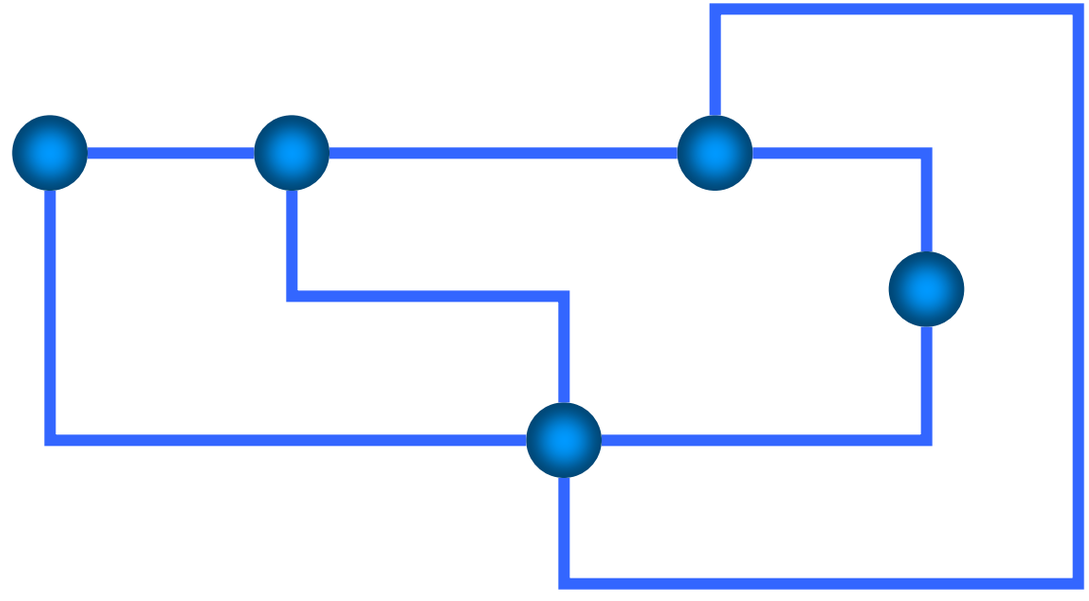
Copyright notice

- All the pages/slides in this presentation, including but not limited to, images, photos, animations, videos, sounds, music, and text (hereby referred to as “material”) are protected by copyright
- This material, with the exception of some multimedia elements licensed by other organizations, is property of the authors and/or organizations appearing in the first slide
- This material, or its parts, can be reproduced and used for didactical purposes within universities and schools, provided that this happens for non-profit purposes
- Any other use is prohibited, unless explicitly authorized by the authors on the basis of an explicit agreement
- This copyright notice must always be redistributed together with the material, or its portions

 copyright ©2025 m.patrignani

Definition of orthogonal drawing

- An *orthogonal drawing* is a drawing of a graph where edges are polylines composed of alternate horizontal and vertical segments



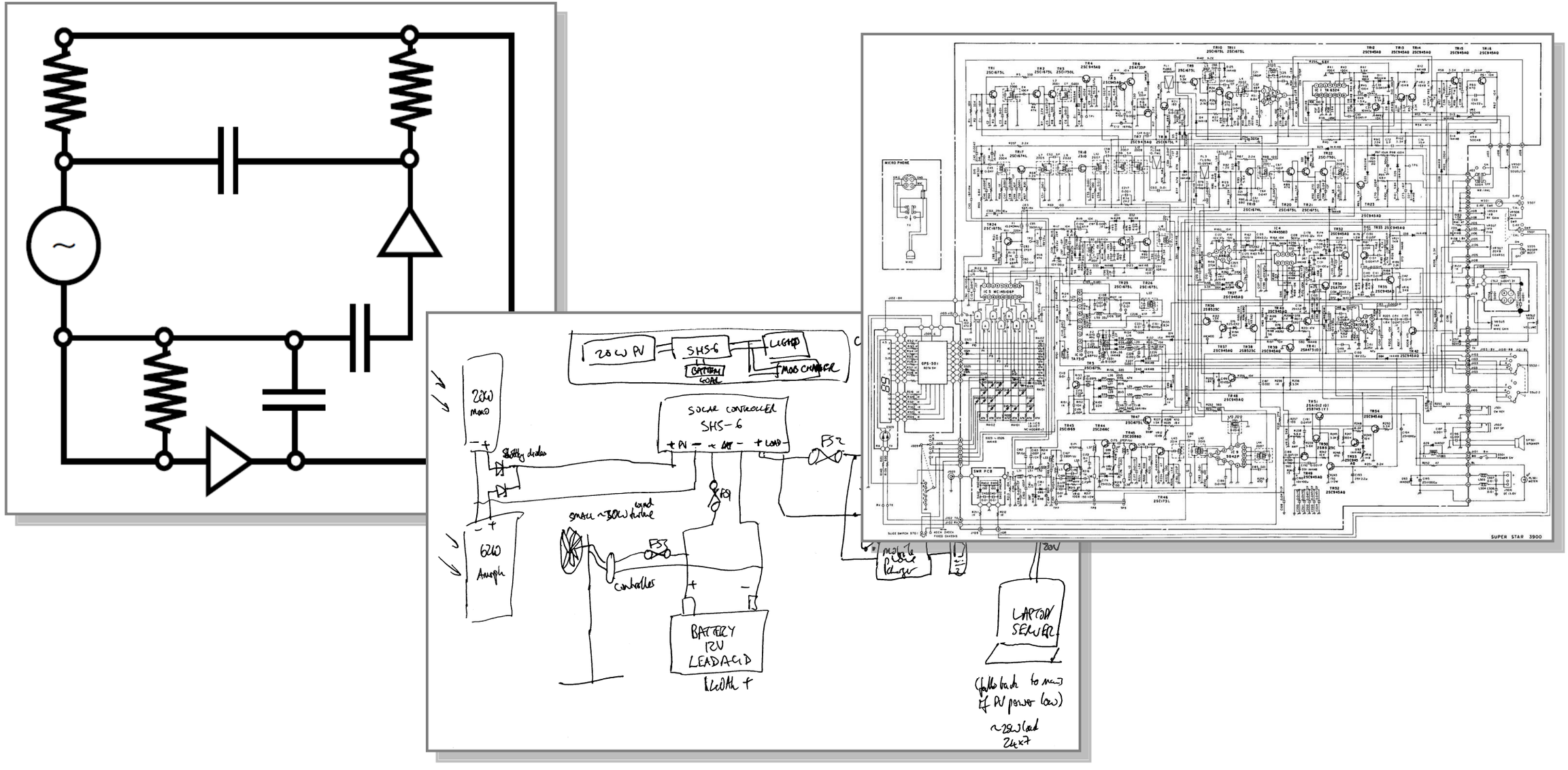
 copyright ©2025 m.patrignani

Orthogonal drawings and applications

- Orthogonal drawings are widely used in software and hardware engineering
- Examples include
  - Electric circuits
  - Entity-Relationship diagrams
  - Relational schemas
  - Data-flow diagrams
  - Flow charts
  - UML diagrams
  - …

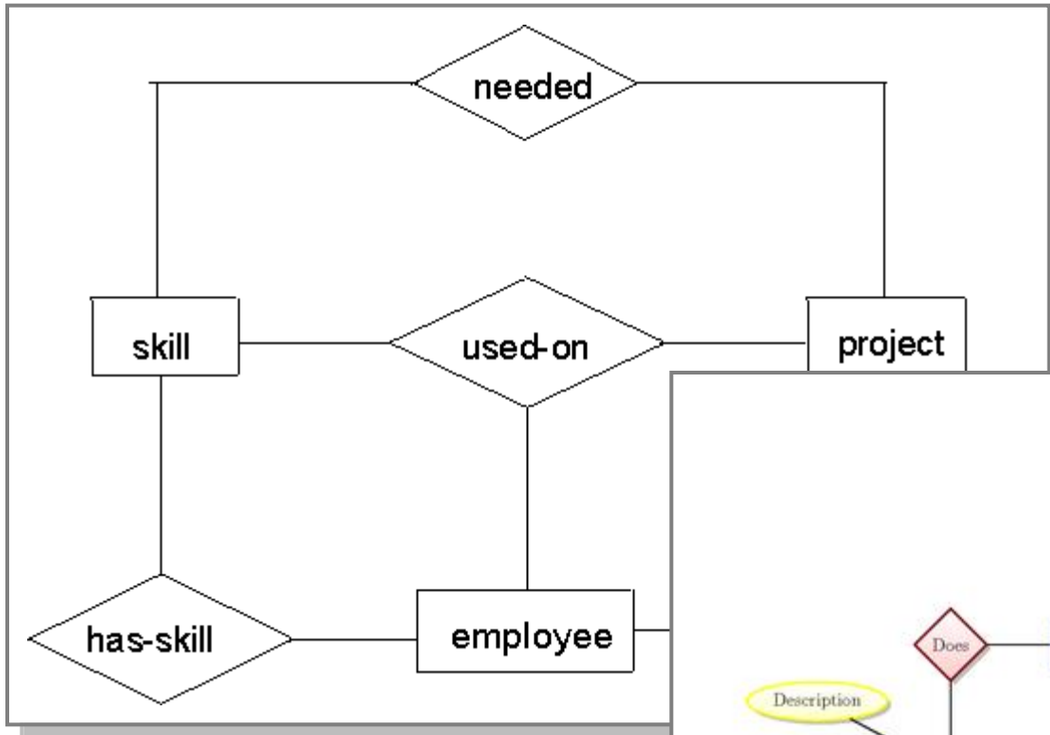
 copyright ©2025 m.patrignani

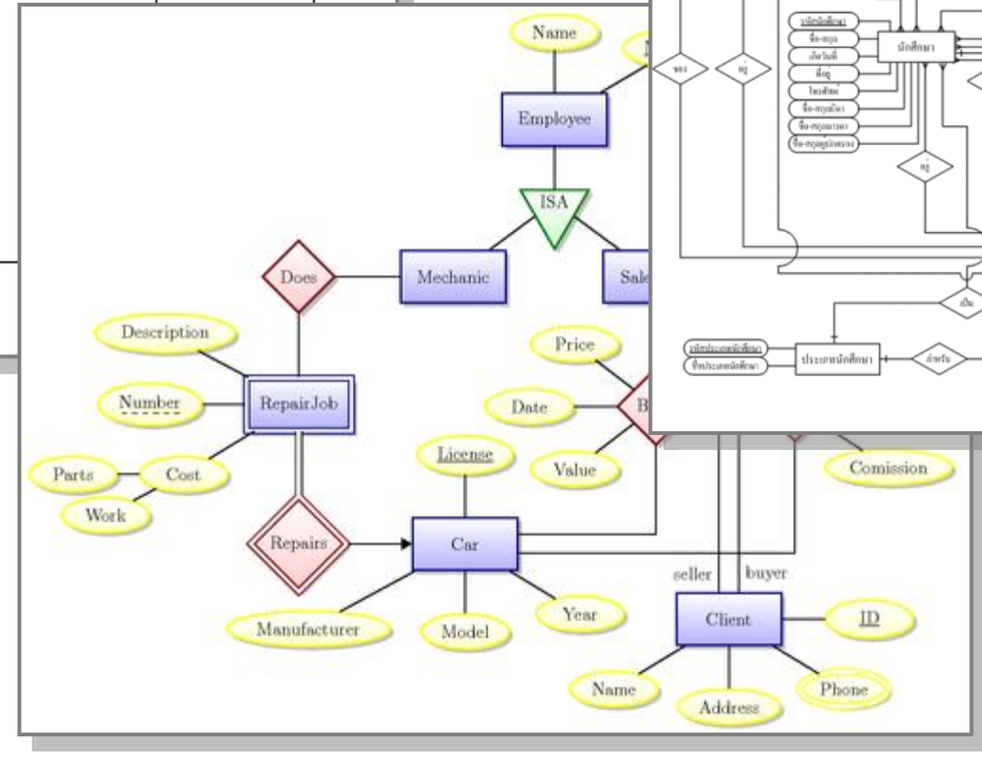
Examples: electric circuits

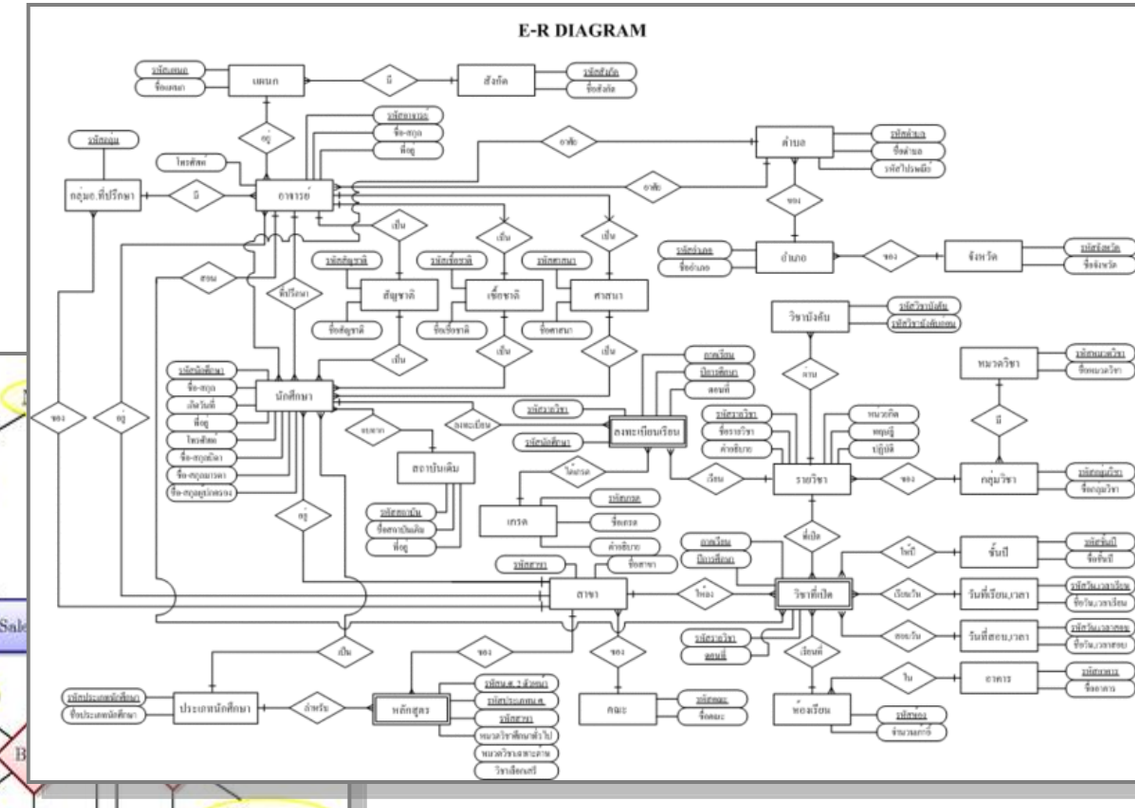


copyright ©2025 m.patrignani

Examples: ER diagrams

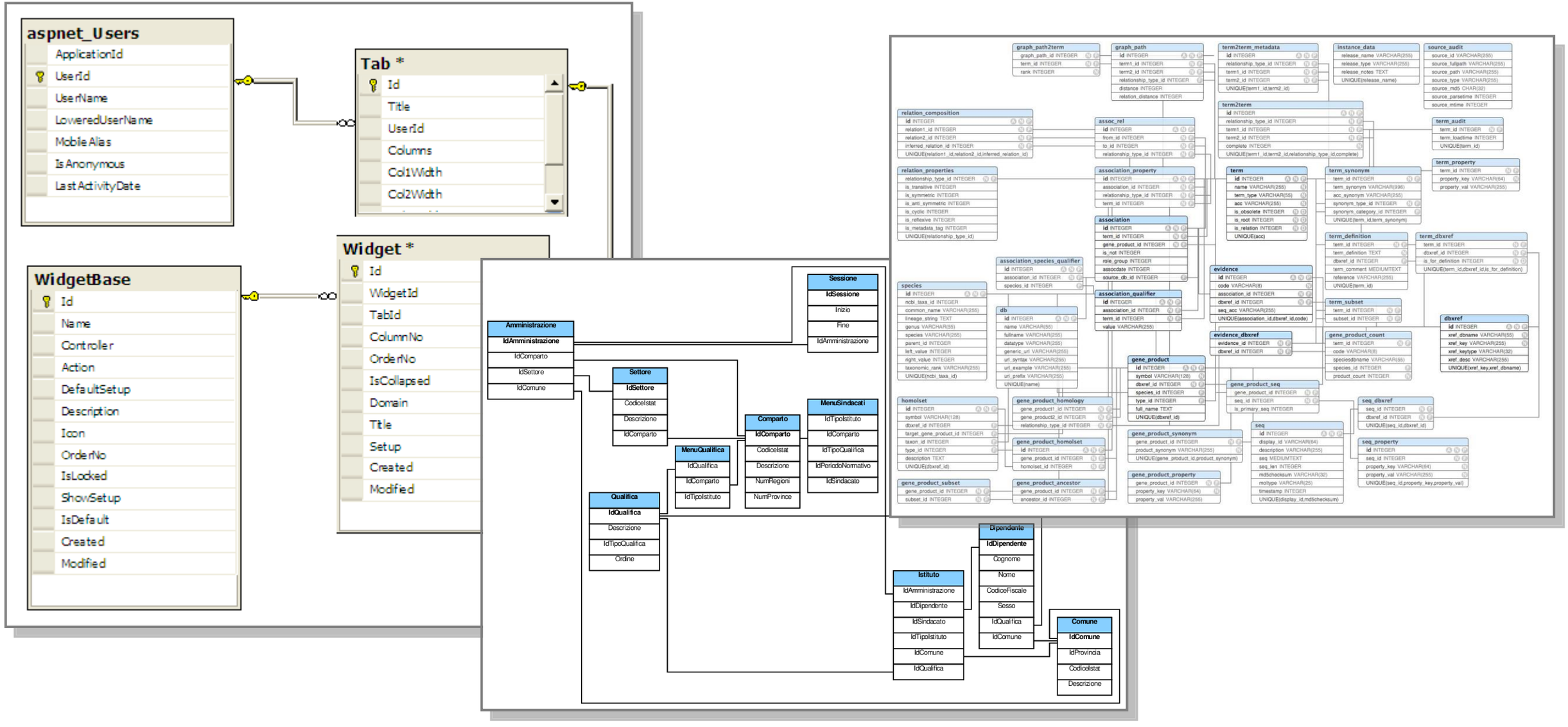






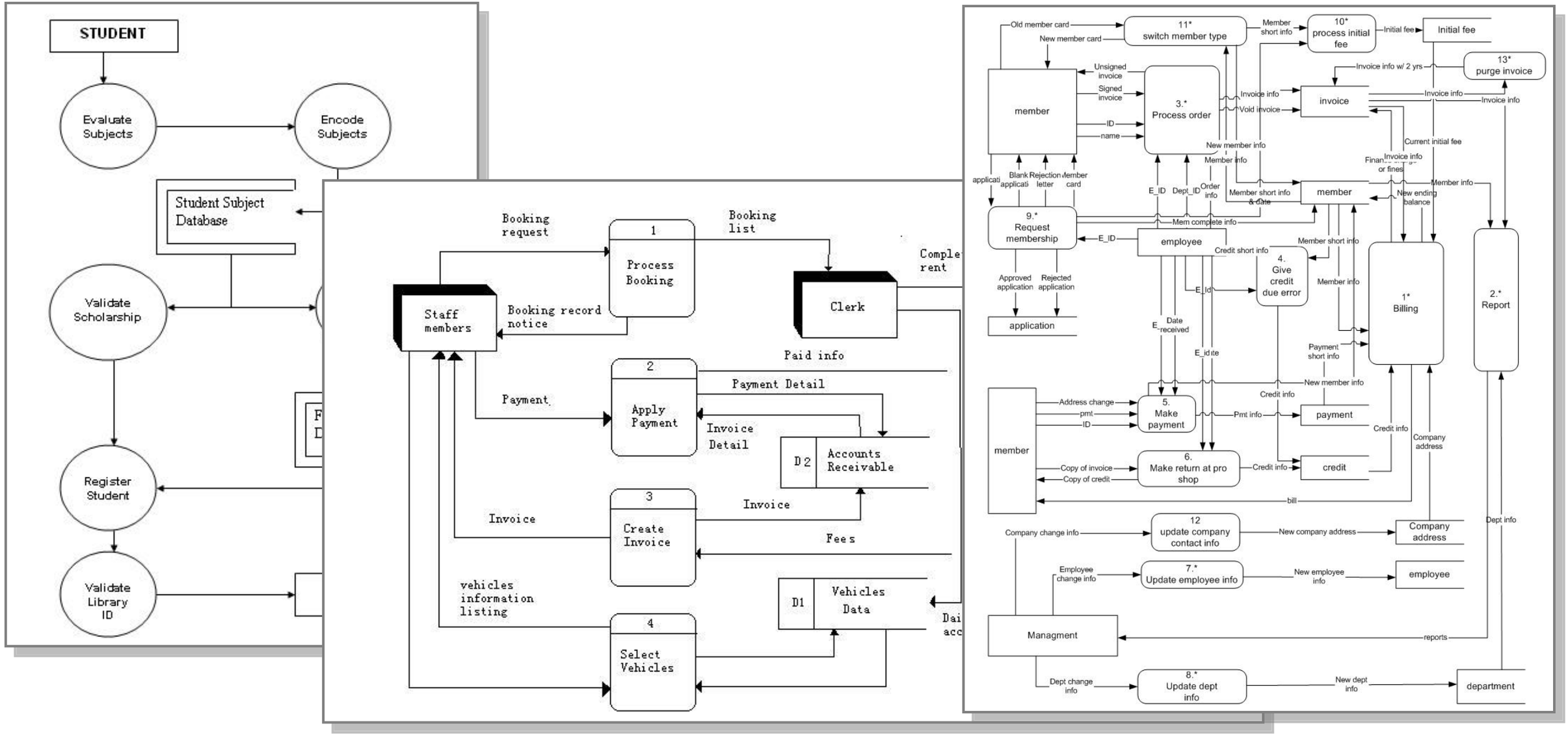
copyright ©2025 m.patrignani

Examples: relational schemas



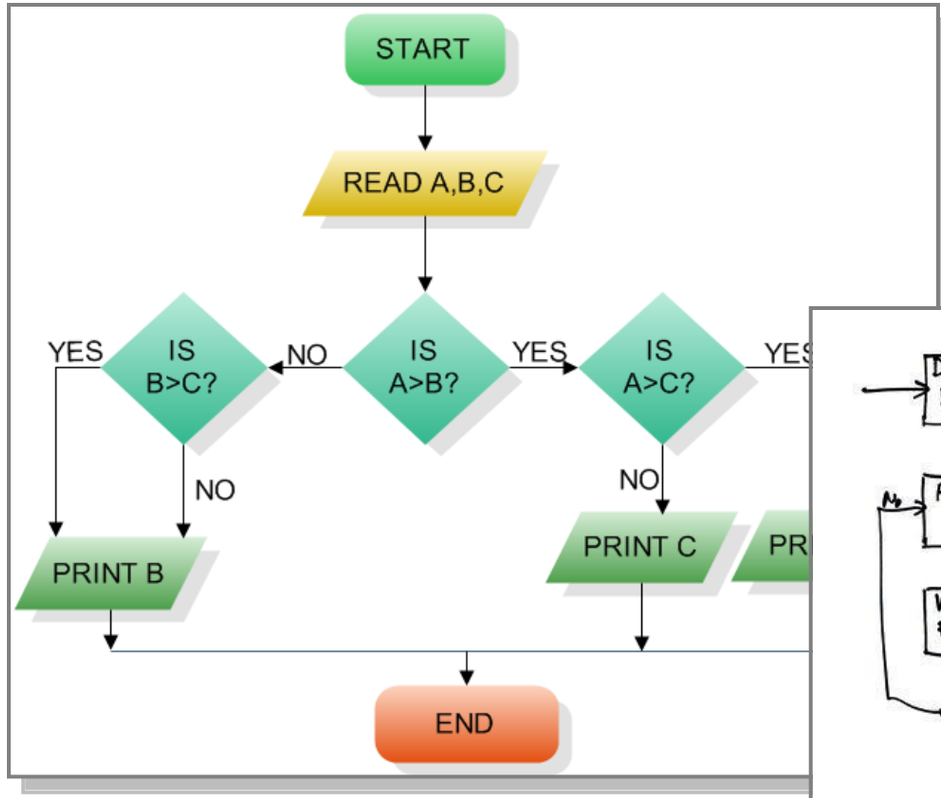
copyright ©2025 m.patrignani

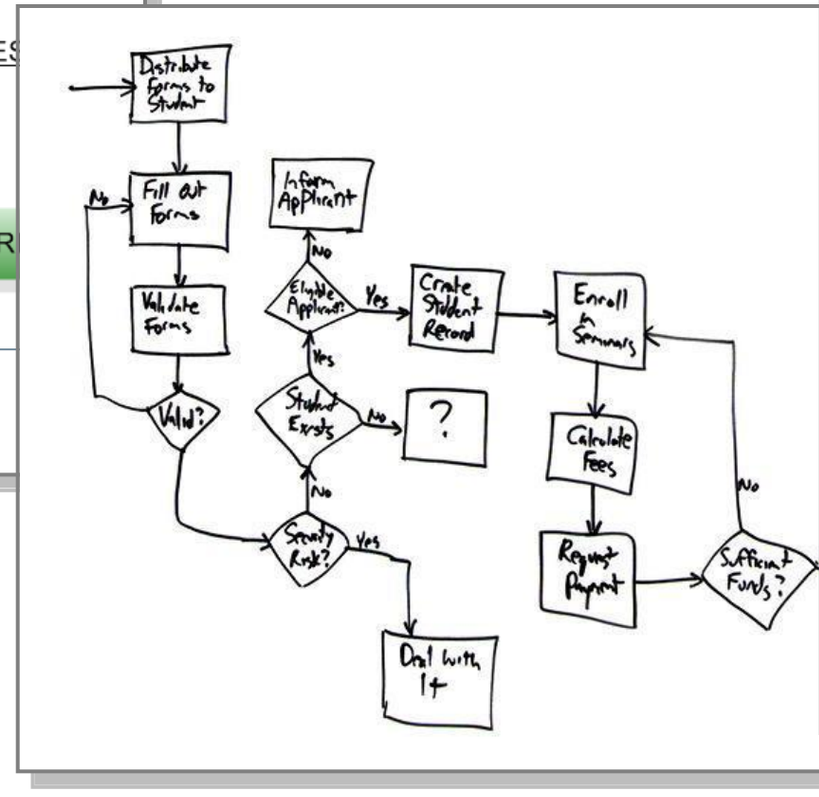
Examples: data-flow diagrams

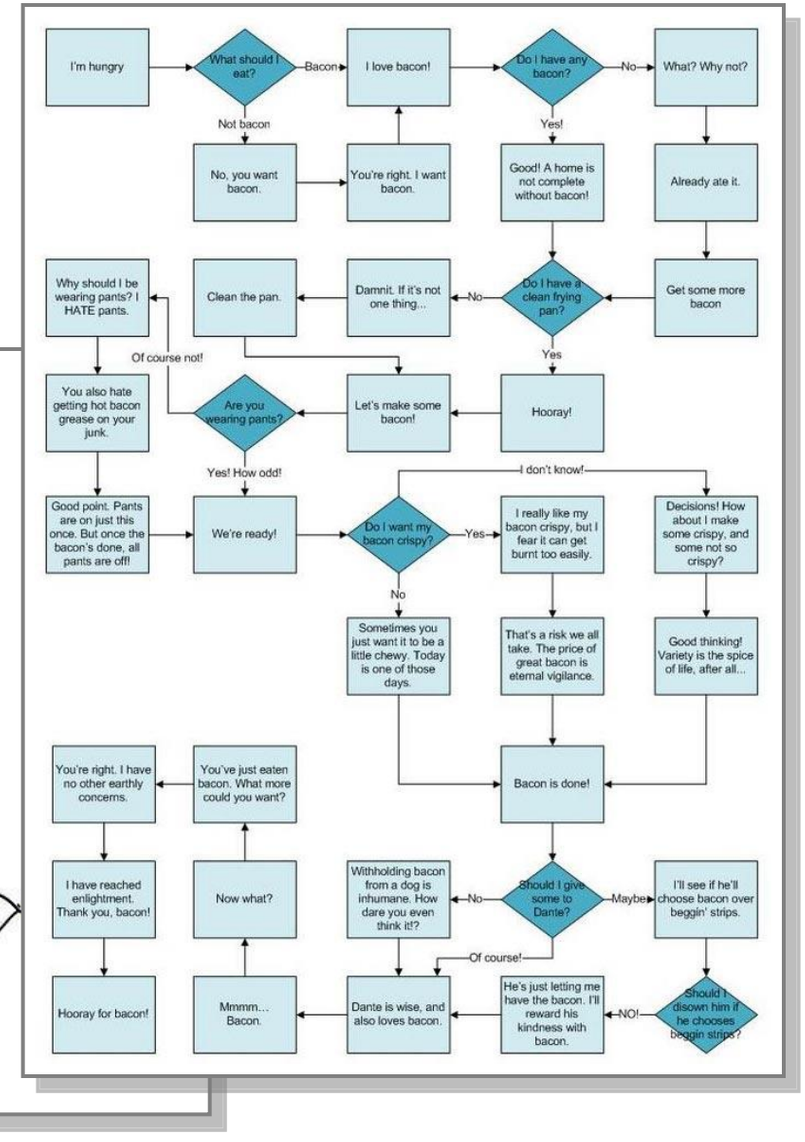


copyright ©2025 m.patrignani

Examples: flow charts

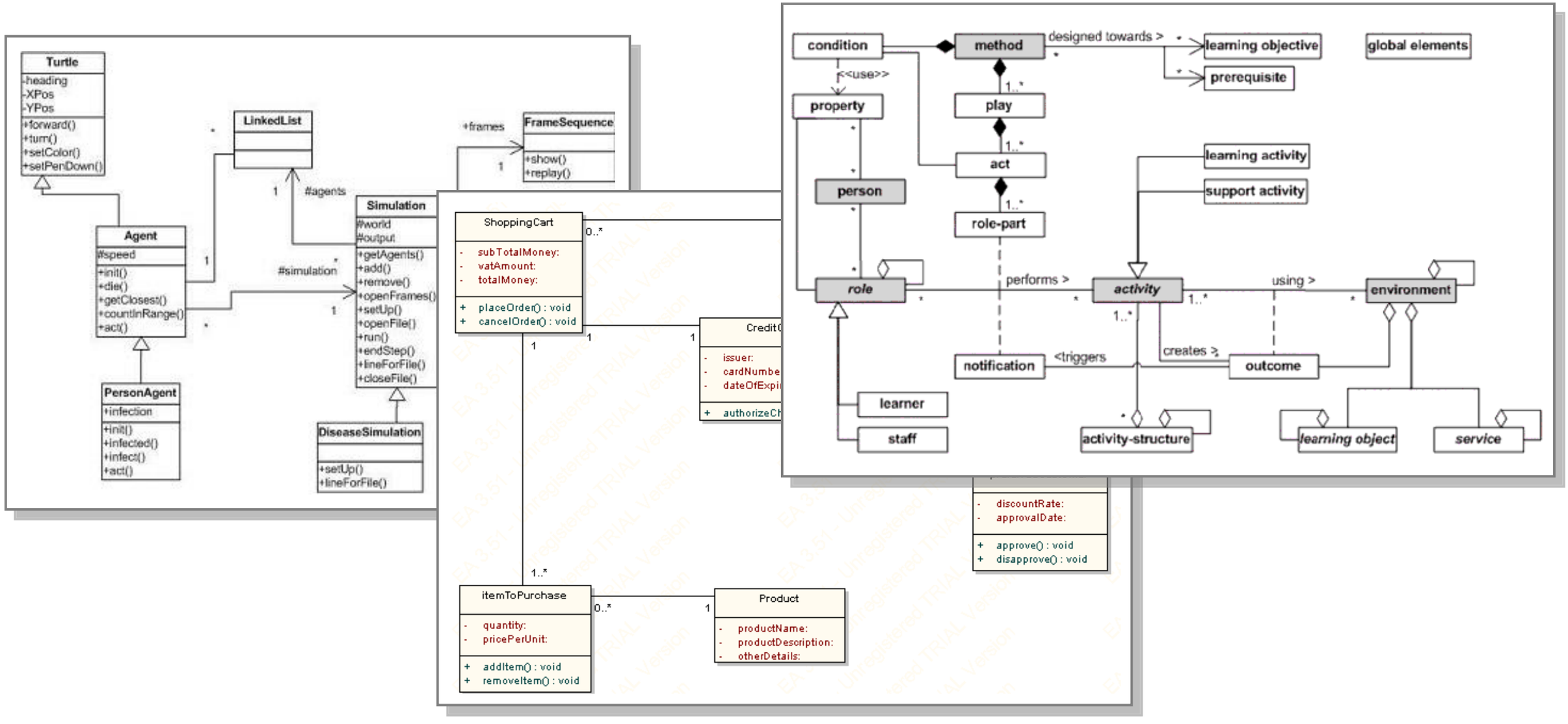






 copyright ©2025 m.patrignani

Examples: UML diagrams



 copyright ©2025 m.patrignani

General problem

Input	A graph G=(V,E)
Output	An orthogonal drawing of G
Target	Minimize some aesthetic measure

- Aesthetics that are usually considered
  - minimize the number of crossings
    - possibly zero crossings is the graph allows it
  - minimize the number of bends
    - minimize the total number of bends
    - minimize the maximum number of bends per edge
  - minimize the size of the drawing
    - minimize the area
    - minimize the maximum edge length
    - minimize the average edge length

copyright ©2025 m.patrignani

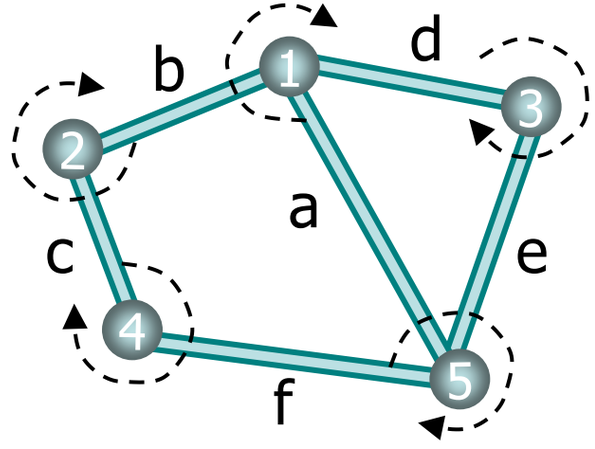
Basic definitions

- A *planar graph* is a graph that admits a planar drawing
  - we already discussed an algorithm to determine whether a graph is planar and to compute a planar embedding of it
- Several planar drawings of the same graph have the same “appearance” or “topology”
  - this concept is captured by the definition of planar embedding

 copyright ©2025 m.patrignani

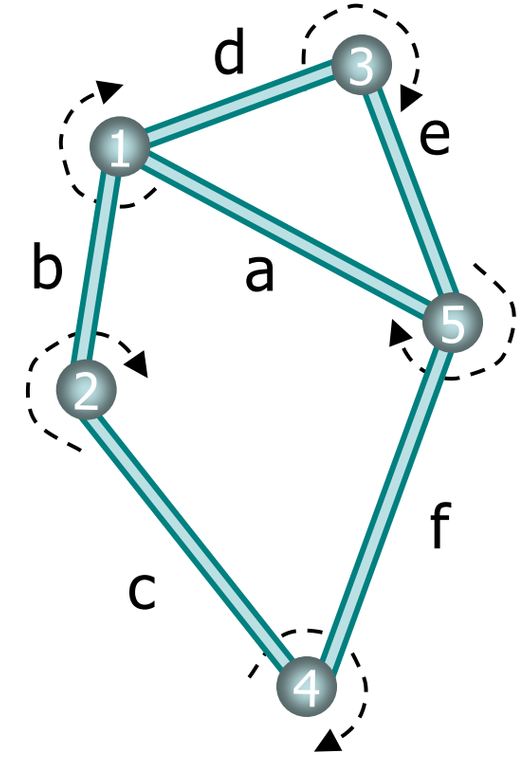
Recall: rotation scheme

- Given a drawing of a graph, the circular order of the edges around each vertex is called *rotation scheme*
- These two drawings have the same rotation scheme



Rotation scheme

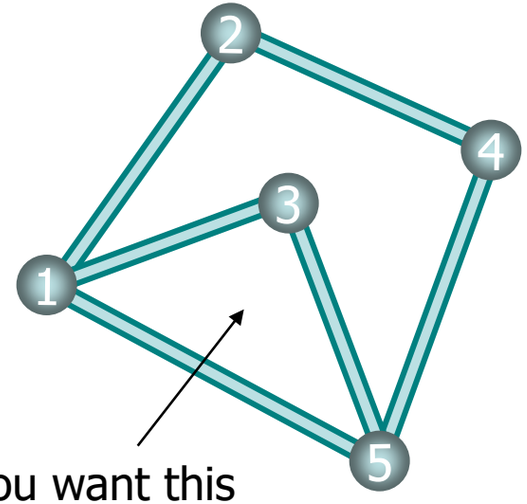
1: a, b, d
2: b, c
3: d, e
4: c, f
5: f, a, e



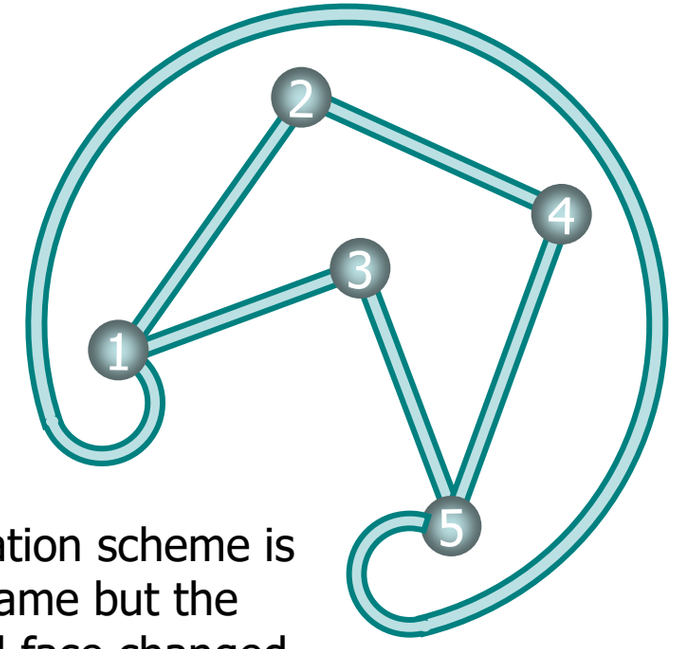
 copyright ©2025 m.patrignani

Recall: external face

- Any face of the graph could be made the external one preserving the same rotation scheme



You want this face to be the external one?



The rotation scheme is the same but the external face changed

 copyright ©2025 m.patrignani

Planar embedding

- A *planar embedding* fixes both the rotation scheme and the external face
  - hence, it is specified by
    - the circular order of the edges around each node
    - the specification of the external face
- A *plane graph* is a graph provided with a planar embedding

 copyright ©2025 m.patrignani

Orthogonal drawing algorithms

- Several approaches are known
  - orthogonal drawings via network flows
  - orthogonal drawings via visibility representations
  - incremental algorithms
- Usually the drawing algorithms are first restricted to handle graphs of maximum degree four
  - then, they are extended to graphs of arbitrary degree

 copyright ©2025 m.patrignani

Orthogonal drawings via network flows

- One of the key results in Graph Drawing
  - solves a problem that was conjectured to be NP-hard
  - offers a nice theoretic model
  - provides an efficient algorithm
  - inaugurates the Graph Drawing research area

 copyright ©2025 m.patrignani

Roberto Tamassia

- Laurea from La Sapienza in 1984
- Ph.D. from Urbana-Champaign in 1988
- Professor at Brown University
  - Plastech Professor of CS
  - Chair of the CS Department
- Research activity
  - more than 200 publications
  - Graph Drawing
  - Computational Geometry
  - Computer Security
- Books
  - Graph Drawing
  - Algorithms and Data Structures
  - Security
  - ...



copyright ©2025 m.patrignani

Topology-Shape-Metric approach

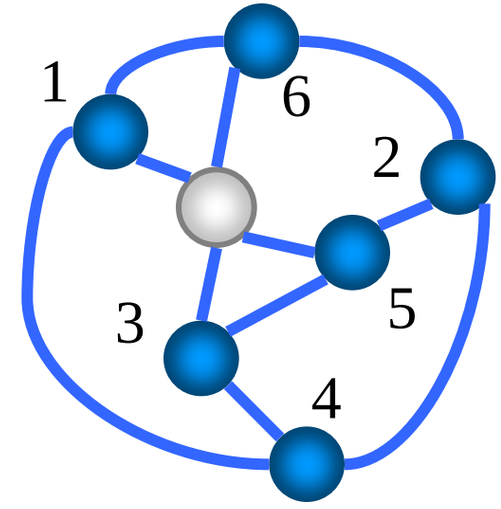
- A three steps method to obtain orthogonal drawings
  - Planarization
    - the topology (planar embedding) of the graph is computed
    - dummy nodes are introduced to replace crossings
  - Orthogonalization
    - the orthogonal shape of the (plane) graph is computed
  - Compaction
    - actual coordinates are computed for nodes and bends
    - dummy vertices are replaced back with crossings

 copyright ©2025 m.patrignani

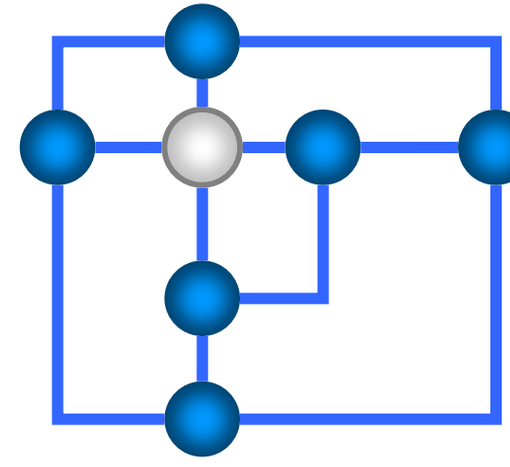
Topology-Shape-Metric example

V={1,2,3,4,5,6}
E={(1,4),(1,5),
(1,6),(2,4),
(2,5),(2,6),
(3,4),(3,5),
(3,6)}

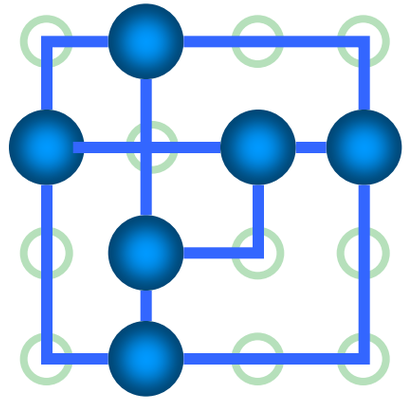
Planarization



Orthogonalization



Compaction



copyright ©2025 m.patrignani

Planarization step

Input	A graph G=(V,E)
Output	A plane graph G' obtained from G by replacing crossings with dummy nodes
Target	Minimize the number of crossings

- Unfortunately crossing minimization is a renown NP-complete problem
  - [Garey & Johnson 83]
- However, the problem is linear-time solvable if G admits a drawing with zero crossings
  - see, for example, [Hopcroft & Tarjan 74] and [Mehlhorn & Mutzel 96]
  - we will use this problem as a subroutine

 copyright ©2025 m.patrignani

A planarization heuristic

- If the graph is planar
  - compute a planar embedding with the algorithm in [Mehlhorn & Mutzel 96]
- If the graph G is not planar
  - start from a spanning tree of G
    - a tree is always planar
  - try to add the remaining edges one at a time
    - if the addition of the edge leaves the graph planar add it to the graph
    - if the addition of the edge would make the graph non-planar save the edge to a set S of “crossing edges”
  - add the “crossing edges” one at a time reducing as much as possible the crossings required by each edge

 copyright ©2025 m.patrignani

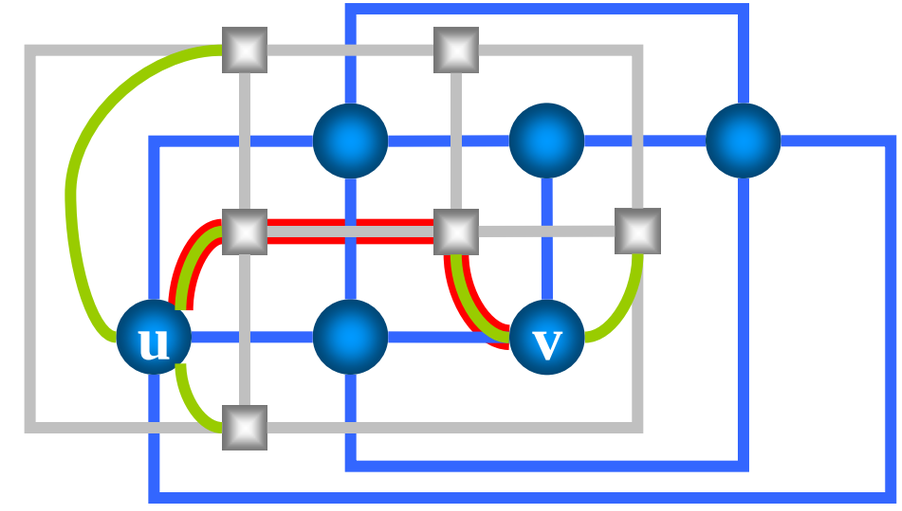
A planarization heuristic

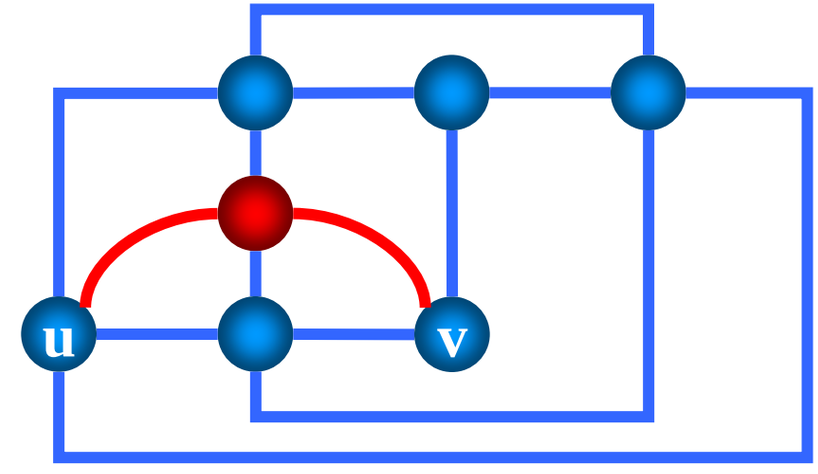
- The addition to the embedding of one "crossing edge" (*u*,*v*) can be done as follows
  1. compute the dual graph of the current planar graph
  2. add *u* and *v* to the dual graph and connect them to their incident faces with dummy edges
  3. search for a shortest path from *u* to *v*

 copyright ©2025 m.patrignani

Planarization example

- A planar graph
- Its dual graph
- Dummy edges
  - connect u and v with their faces
- One shortest path from u to v
- Final graph
  - dummy vertices are added to account for crossings





 copyright ©2025 m.patrignani

Orthogonalization step

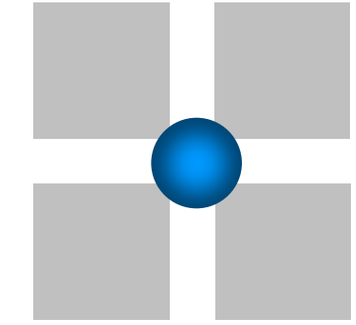
Input	A plane graph G=(V,E)
Output	An orthogonal shape of G
Target	Minimize the number of bends

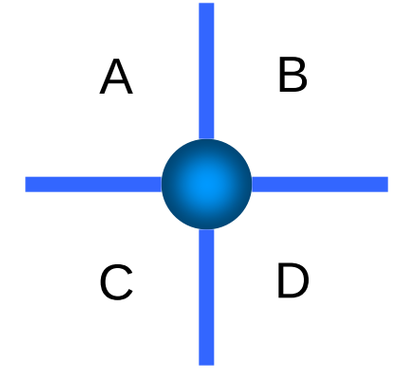
- This problem was thought to be NP-hard ([Storer 80]) but it is, in fact, polynomial
  - [Tamassia 87]

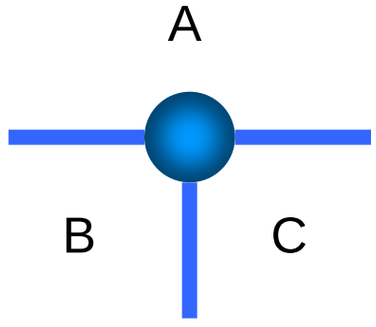
copyright ©2025 m.patrignani

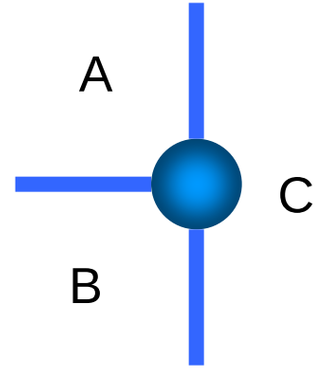
Nodes and right angles

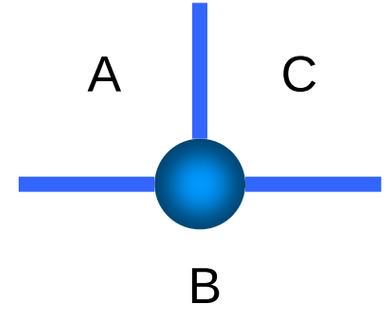
- Each node is surrounded by four right angles
- If the node has degree four, then its drawing, up to a rotation, is already decided
- If the node has degree less than four, several choices are possible
  - for example these are the possible choices for a node of degree three







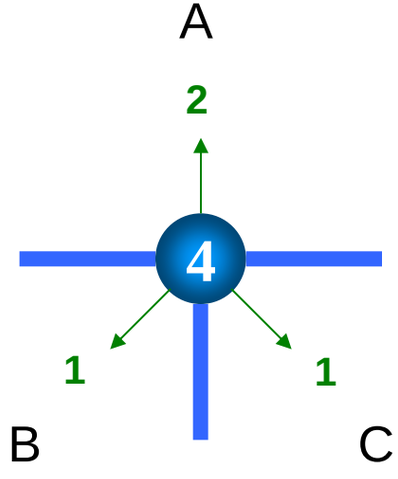


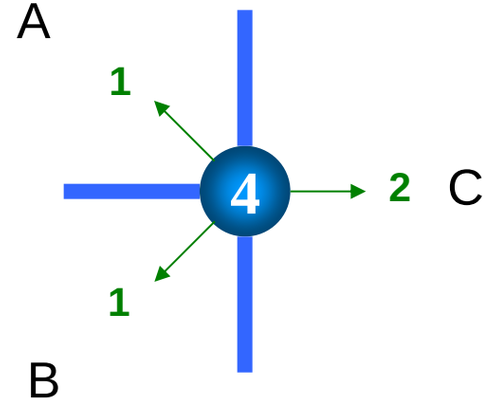


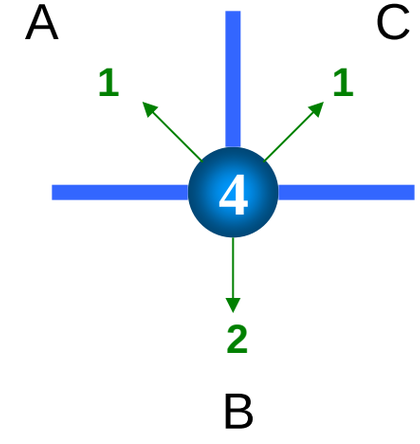
copyright ©2025 m.patrignani

Nodes and flows

- Each right angle may be associated with a unit of flow
  - different drawings correspond to different ways of these four units (right angles) to “flow” out of the nodes and into its incident faces



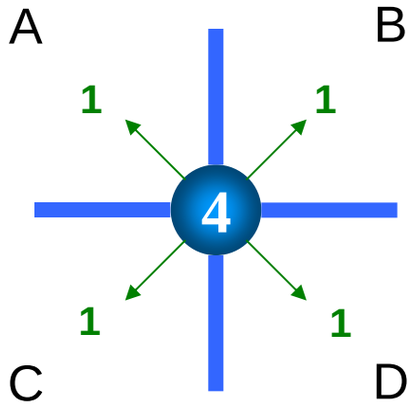


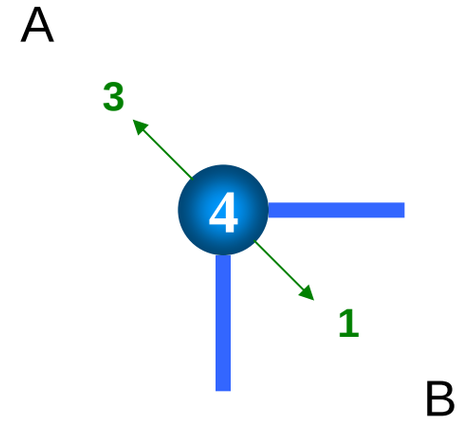


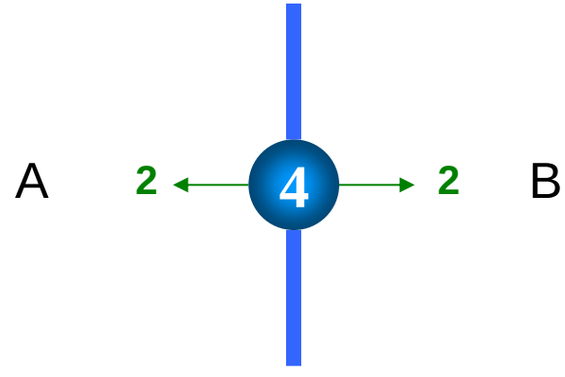
copyright ©2025 m.patrignani

Nodes and flows

- The same is true also for nodes of degree four, two, or one





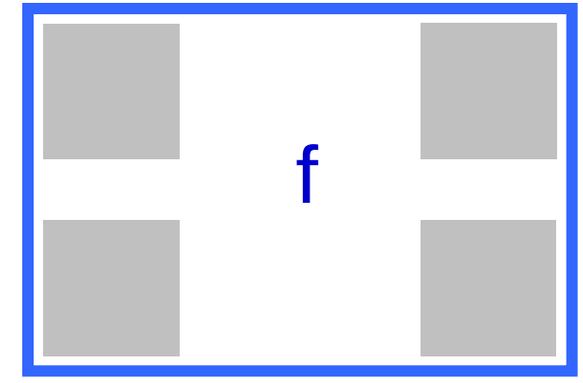


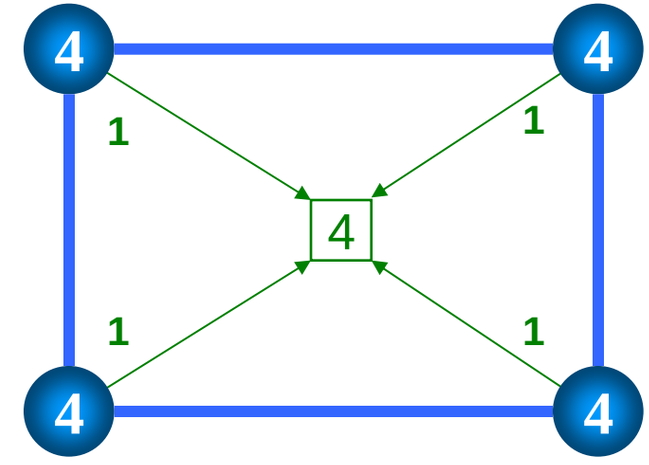


 copyright ©2025 m.patrignani

Rectangular faces and flows

- Let's focus on a rectangular face
  - a face that has the shape of a rectangle
  - call deg(f) is the number of nodes encountered while traversing the boundary of face f
- if deg(f) = 4 the face needs 4 units of flow
  - 4 right angles

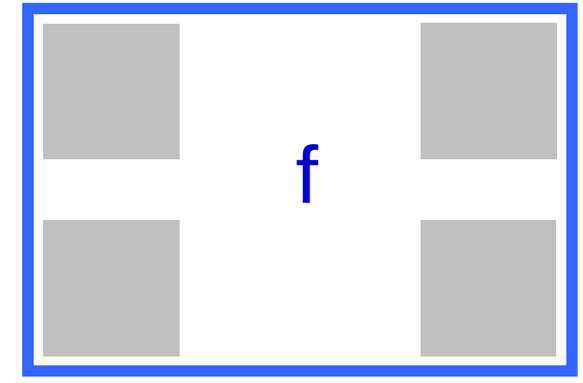


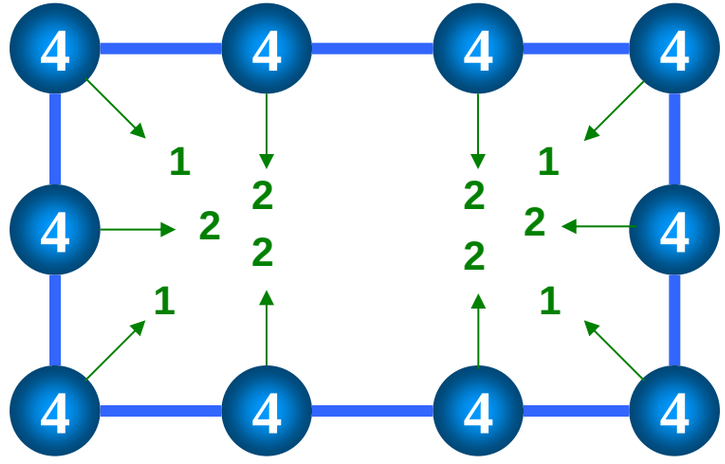


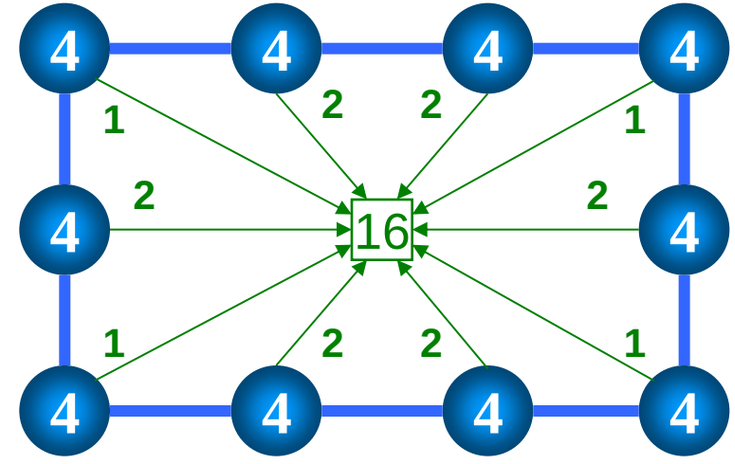
 copyright ©2025 m.patrignani

Rectangular faces and flows

- Let's add more nodes to the rectangular face
- If f has deg(f) nodes, it takes 2deg(f)-4 units of flow
  - hypothesis: each face always takes 2deg(f)-4?



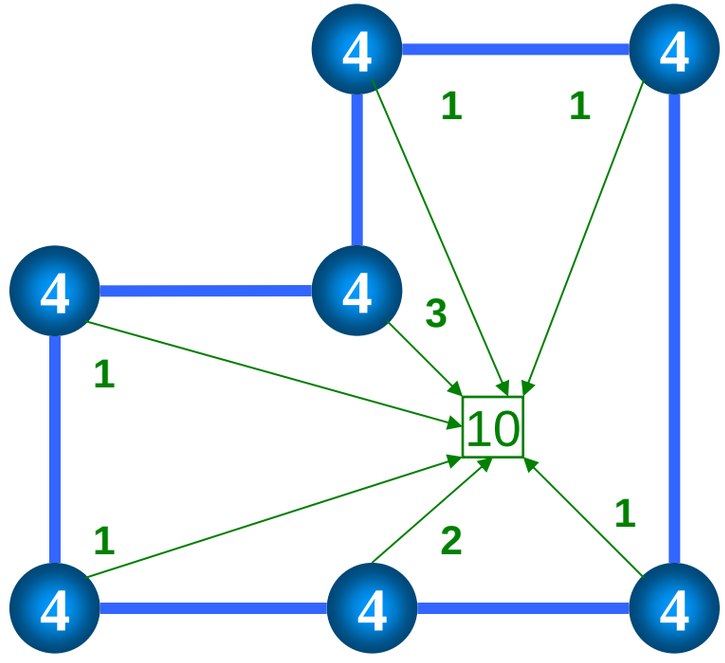


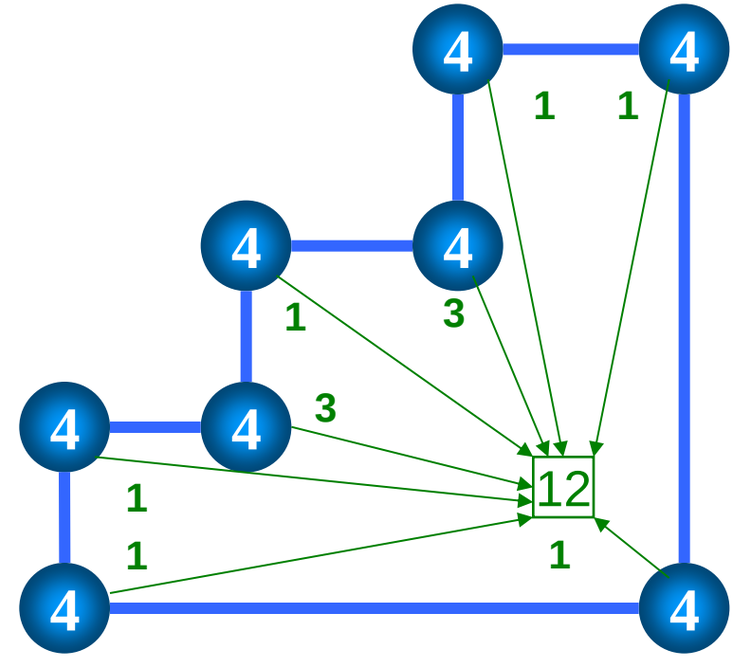


 copyright ©2025 m.patrignani

Faces and flows

- The formula 2deg(f)-4 also holds for any (internal) face whatever is its shape
  - but we are not considering bends yet

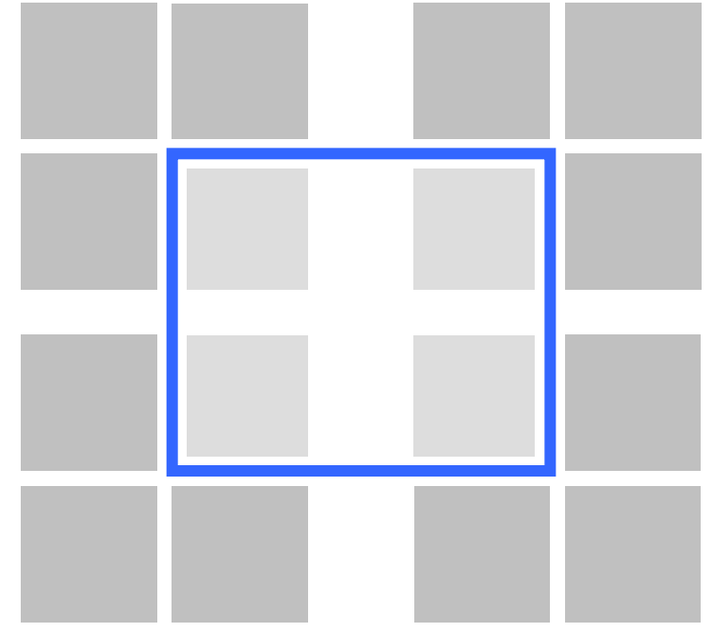


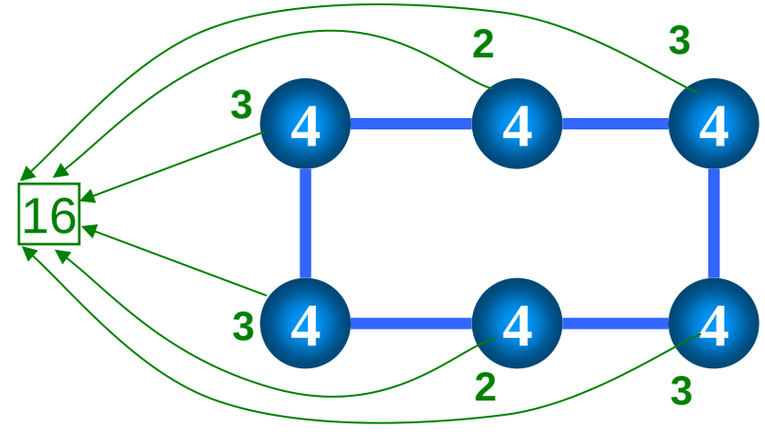


 copyright ©2025 m.patrignani

External face

- The external face is an exception
  - if it is rectangular, instead of using four right angles it needs 12
- For the external face the formula 2deg(f)+4 is used instead of 2deg(f)-4

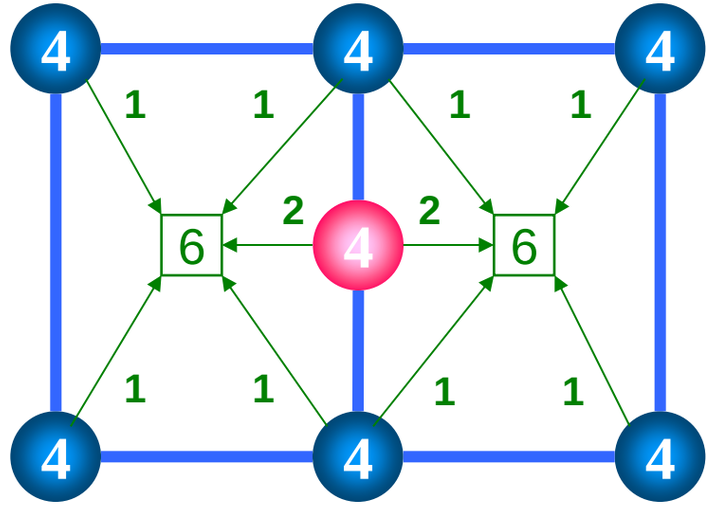


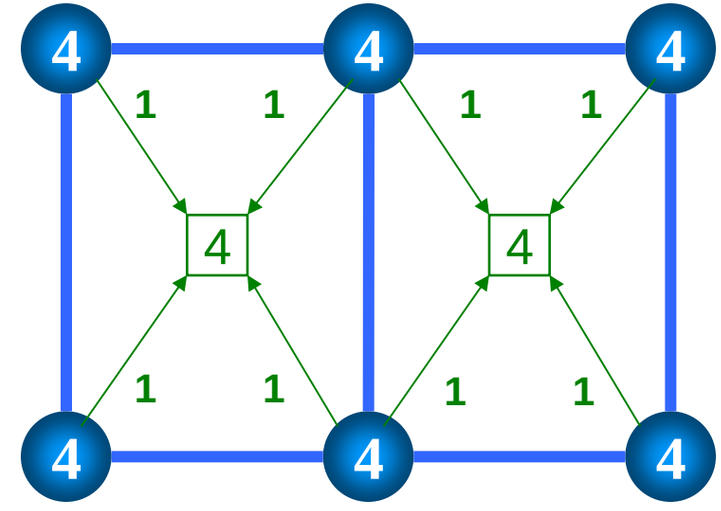


 copyright ©2025 m.patrignani

Bends and flows

- Suppose you remove a degree-two node
  - its incident edges must be joined



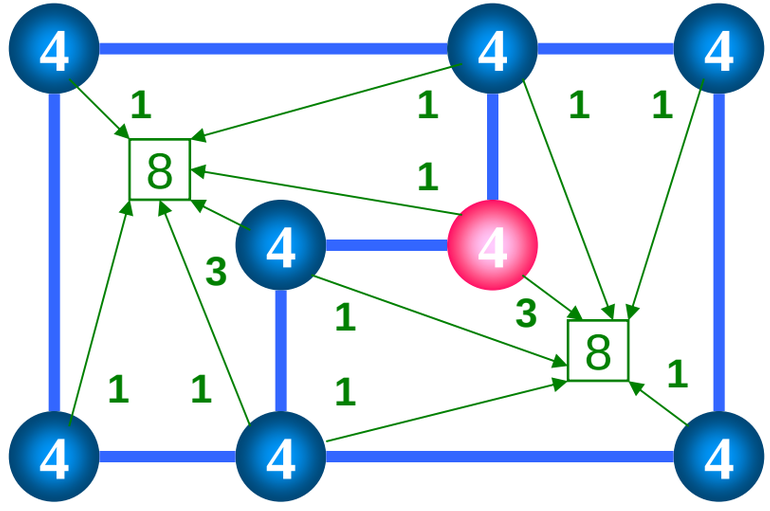


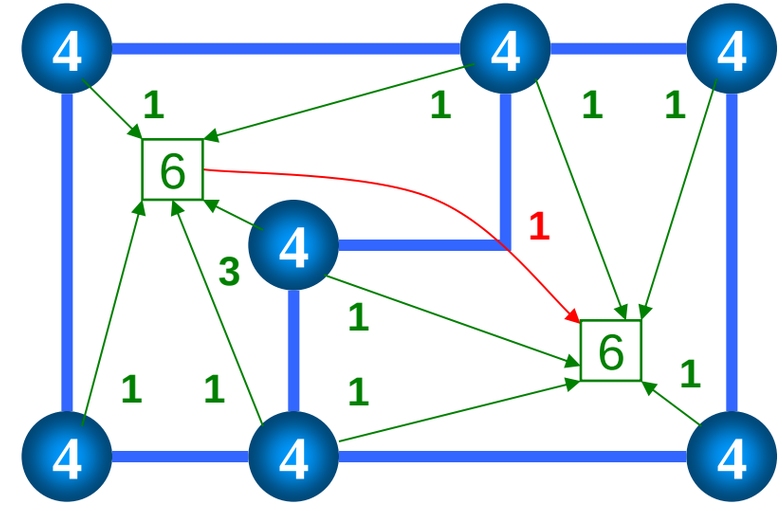
- If the node's flows were 2 and 2 no bend is introduced
  - also, the flow remains balanced

 copyright ©2025 m.patrignani

Bends and flows

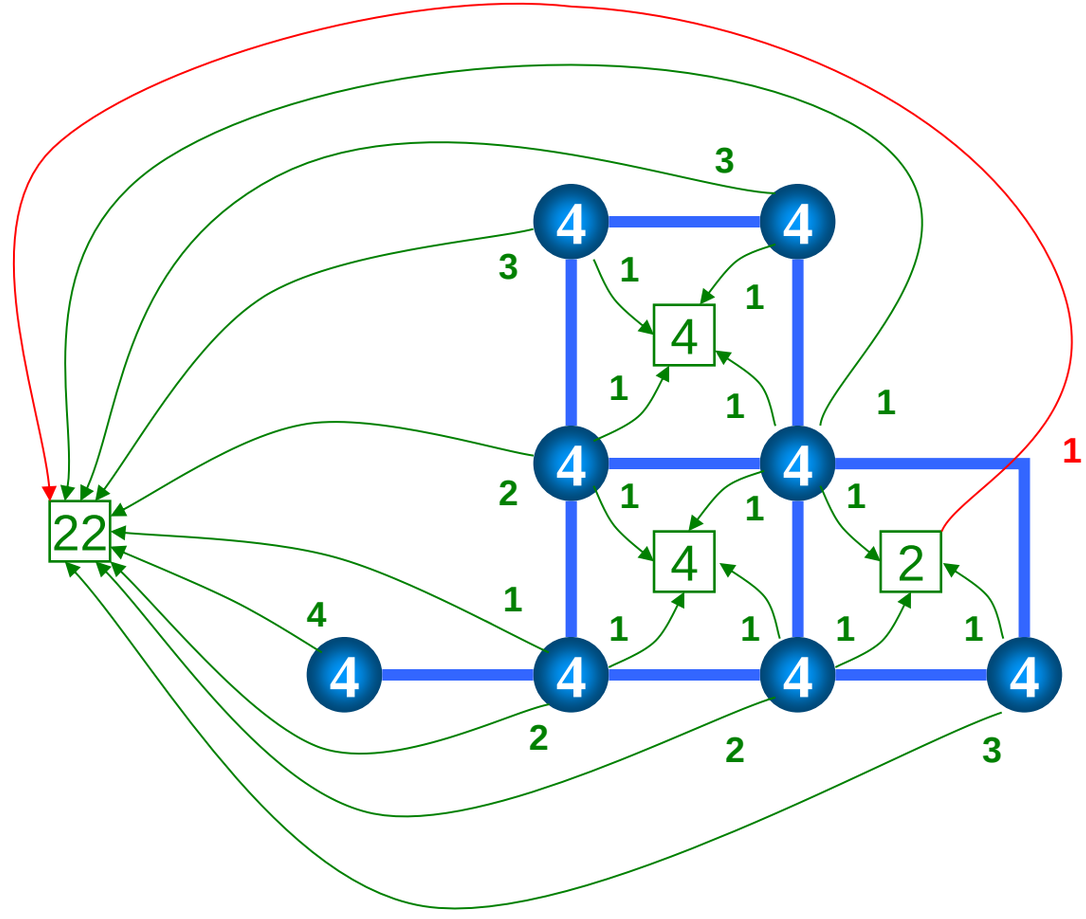
- If the node's flows were 1 and 3
  - one bend is introduced
  - the flow is unbalanced
    - unless one unit of flow traverses the bend!





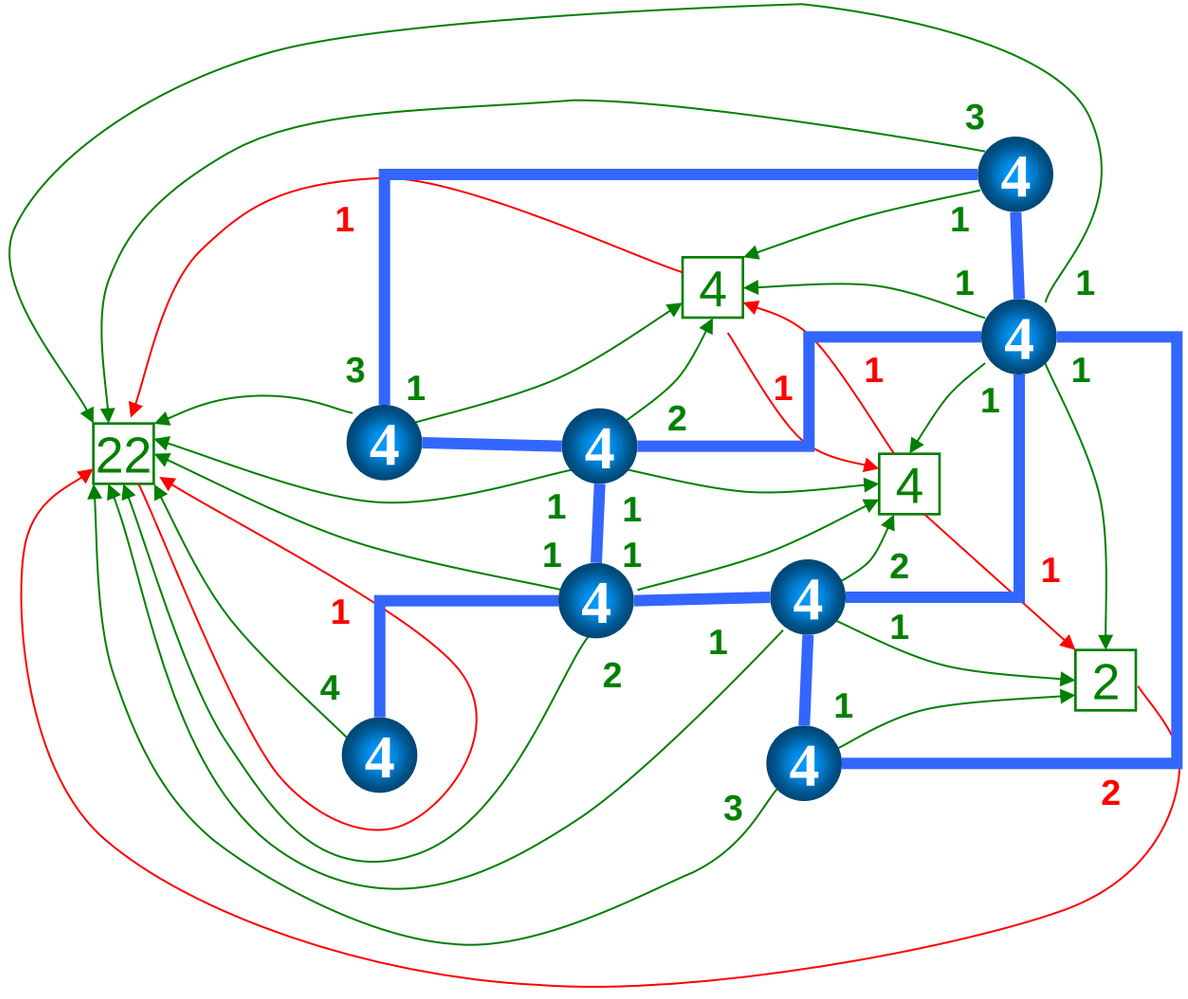
 copyright ©2025 m.patrignani

A whole example



 copyright ©2025 m.patrignani

A more complex example



 copyright ©2025 m.patrignani

Flows and costs

- We associate a cost of one unit to each unit of flow exchanged between adjacent faces
- A drawing that has few bends is a drawing that has an associated flow with reduced cost

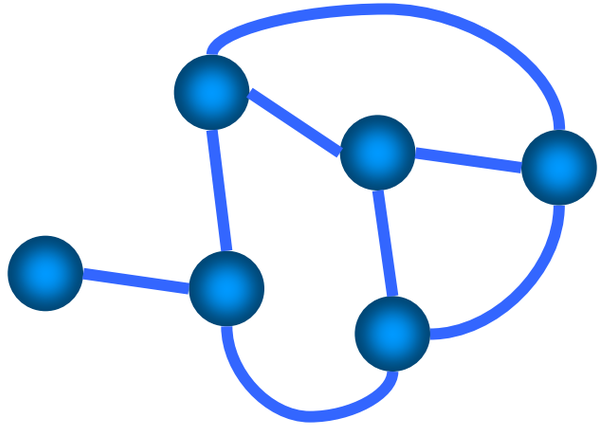
 copyright ©2025 m.patrignani

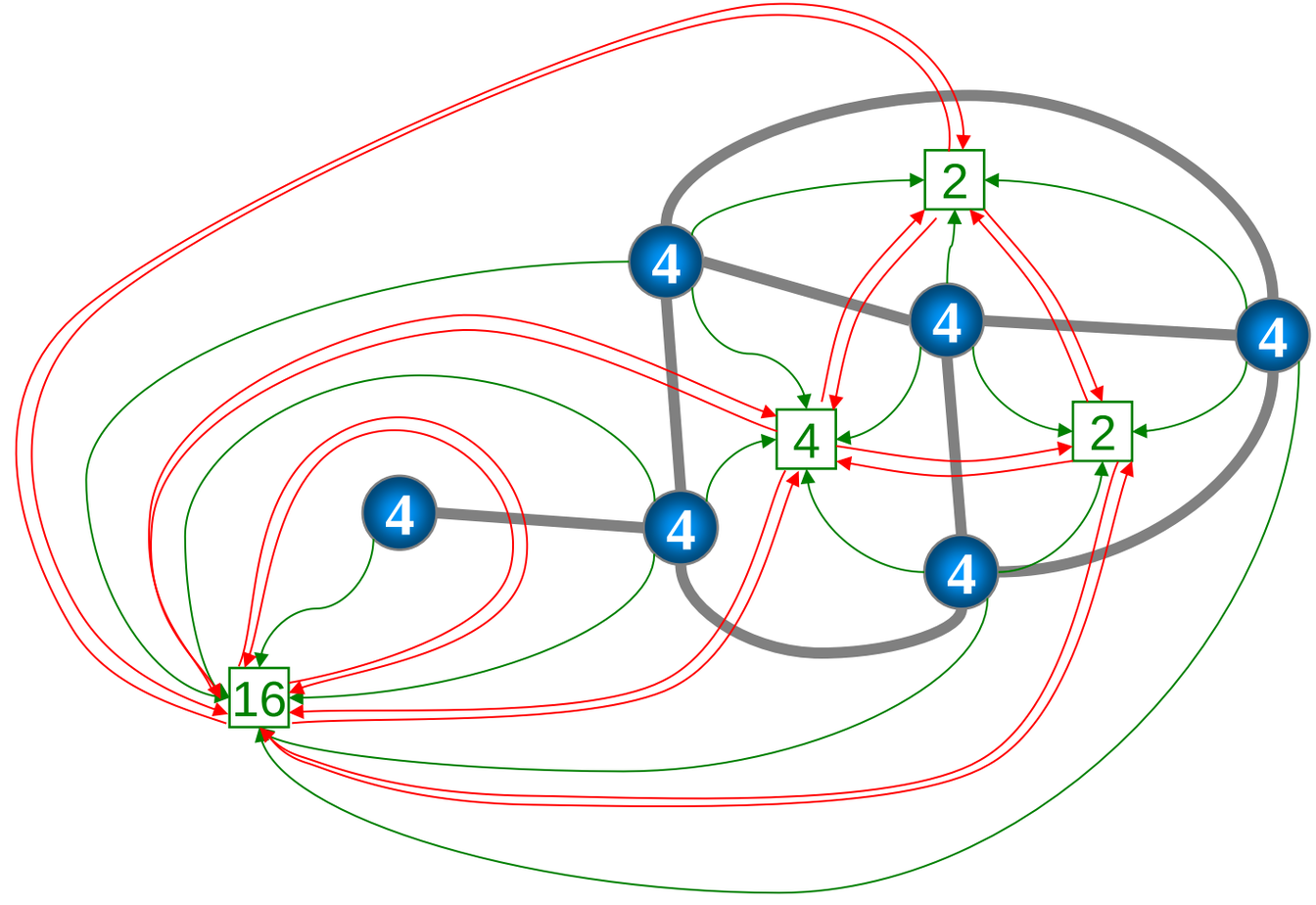
Network flow definition

- Add to N(G) a node n_v for each node v of G
  - n_v is a source of 4 units of flow
- Add to N(G) a node n_f for each face f of G
  - n_f is a sink of $2\deg(f)-4$ units if f is an internal face, otherwise it is a sink of $2\deg(f)+4$ units
- Add to N(G) one arc (n_v, n_f) for each vertex v of G incident to a face f of G
  - (n_v, n_f) has cost 0 and lower bound 1
  - Multiple incidences of the same vertex to the same face yield multiple arcs of N(G)
- Add to N(G) two arcs (n_f, n_g) and (n_g, n_f) for each edge e of G, with adjacent faces f and g
  - both arcs have cost 1 and lower bound 0

 copyright ©2025 m.patrignani

Example of network construction





 copyright ©2025 m.patrignani

Feasible flow

- A *feasible flow* is a flow f(*a*) for each arc *a* of the network N(G) such that
  - flow f(*a*) is at least the lower bound of *a*
  - the sum of the flows exiting *a* node n minus the sum of the flows entering n is
    - equal to k if node n is a source of k units of flow
    - equal to –k if node n is a sink of k units of flow
    - equal to zero otherwise
      - we do not have this kind of nodes
- The *cost* of the flow is the sum of f(*a*)×c(*a*)
  - where c(*a*) is the cost of arc *a*

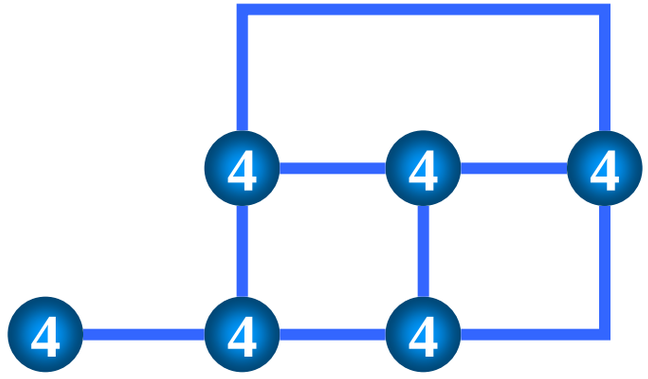
 copyright ©2025 m.patrignani

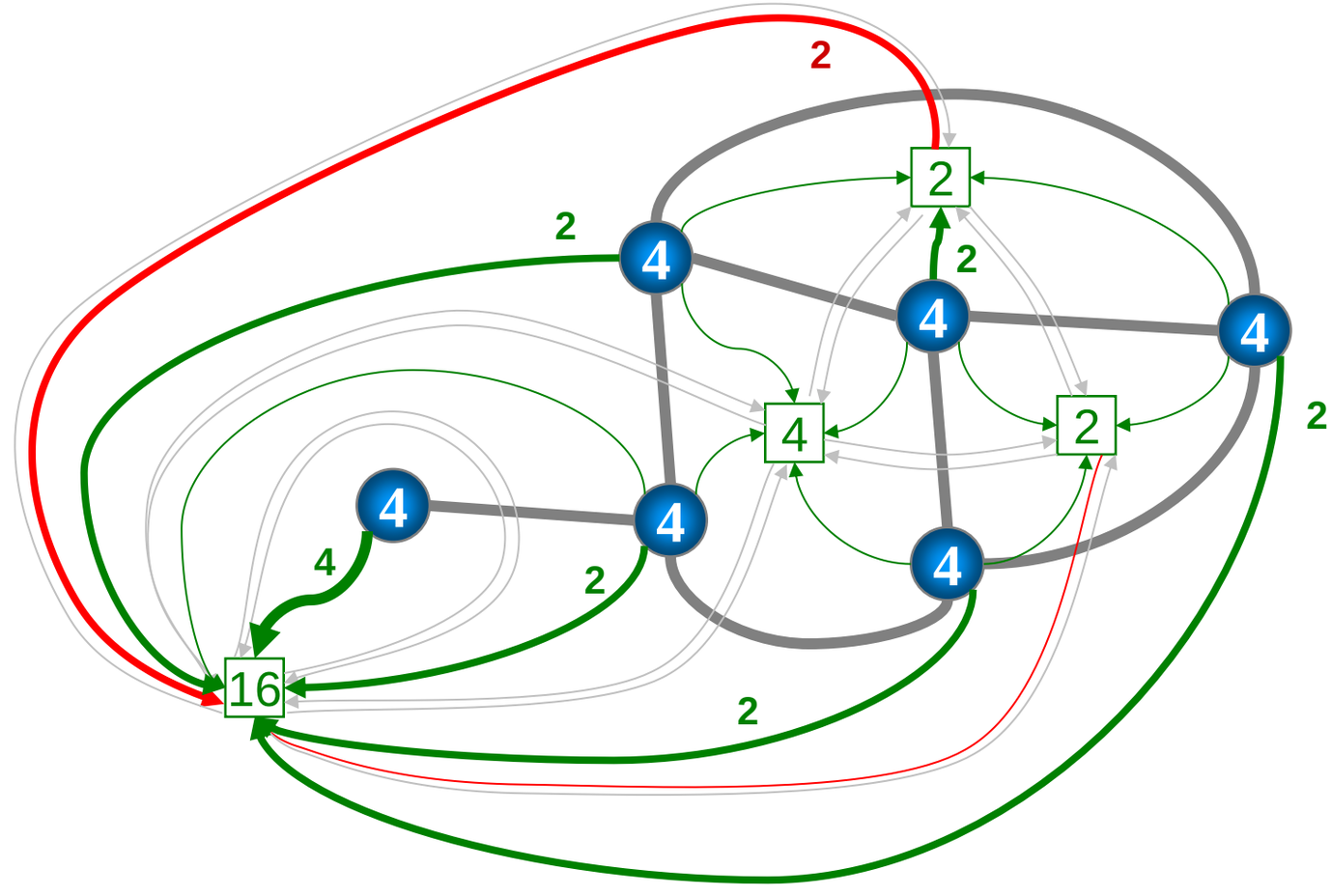
Main result

- Theorem [Tamassia 87]
  - an orthogonal drawing of a plane graph G with the minimum number of bends corresponds to a feasible flow of N(G) with minimum cost
  - given a feasible flow of N(G) with minimum cost, it is possible to compute one orthogonal drawing of G with the minimum number of bends

 copyright ©2025 m.patrignani

Example of minimum cost flow





 copyright ©2025 m.patrignani

Computing a minimum cost flow

- The solution to this problem proposed in [Tamassia 87] is based on an iterative augmentation algorithm
  - start from the initial zero flow
  - compute the final $4 \times n$ flow by augmenting the flow at each of the O(n) steps along a minimum cost path
    - such a path can be found with the O(n log n)-implementation of Dijkstra's algorithm exploiting a priority queue
- The overall $O(n^2 \log n)$ time complexity was lowered first to $O(n^{7/4} (\log n)^{1/2})$ by [Garg & Tamassia, '96]
  - recently, this bound was lowered again to $O(n^{3/2})$ by exploiting the planarity of N(G)
    - [Cornelsen & Karrenbauer, 12]

 copyright ©2025 m.patrignani

Orthogonal representations

- The output of the network flow algorithm is an orthogonal representation of G
  - that is, a description of the “shape” of the drawing
- Formally, an *orthogonal representation* is
  - a description of the angles around each node of G such that
    - each angle is a multiple of $\pi/2$
    - the sum of the angles of the same node is 2π
  - a description of the angles encountered while traversing each edge such that
    - traversing an internal face f you sum up $\pi deg(f)-2$
    - traversing the external face f you sum up $\pi deg(f)+2$

 copyright ©2025 m.patrignani

The compaction step

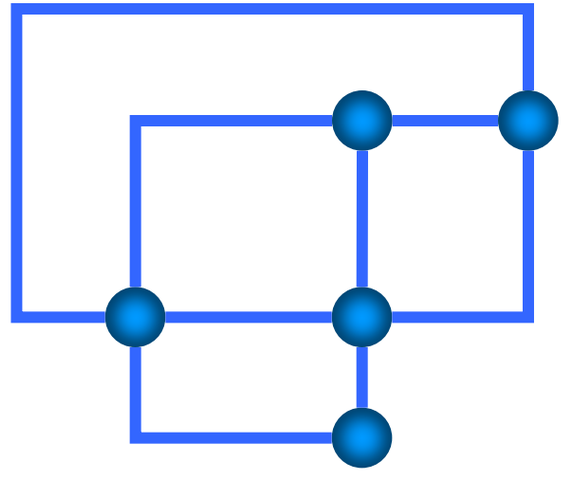
Input	An orthogonal representation of a plane graph G
Output	An orthogonal grid drawing of G
Target	Minimize the size of the drawing

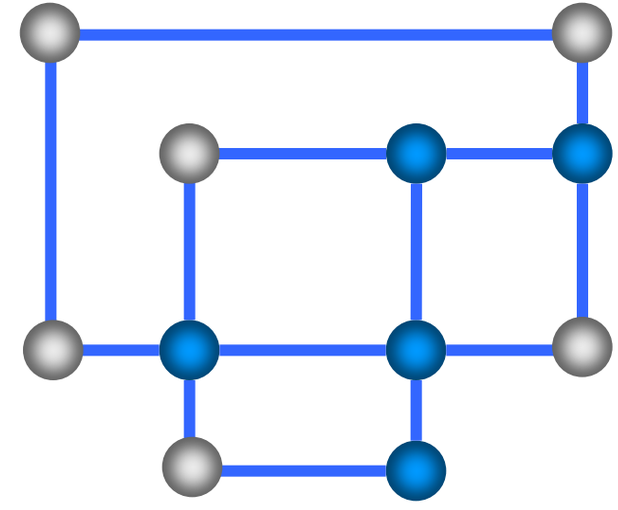
- An *orthogonal grid drawing* of a plane graph G is an orthogonal drawing where nodes and bends have integer coordinates
- Usually you try to minimize
  - the area
  - the maximum edge length
  - the average edge length

 copyright ©2025 m.patrignani

Compaction and bends

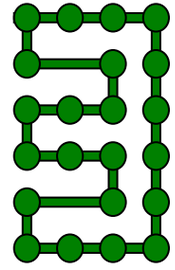
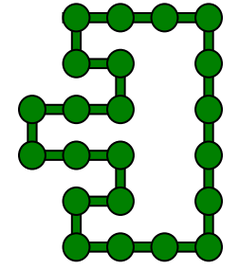
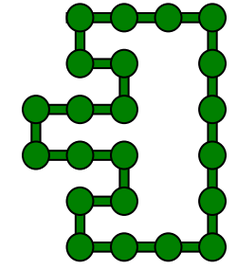
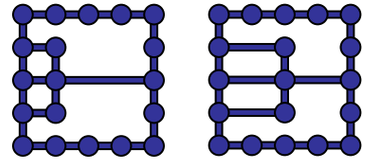
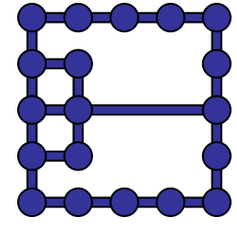
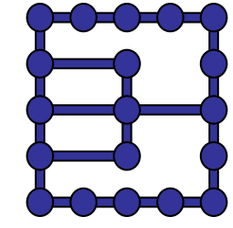
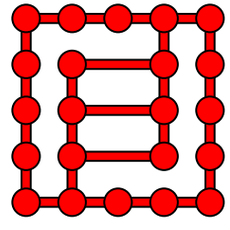
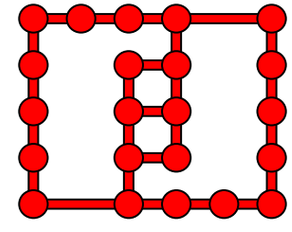
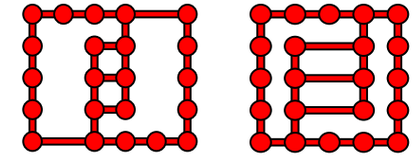
- For the sake of compaction we only consider orthogonal representations without bends
- Otherwise
  - replace bends with dummy nodes
  - compact the obtained drawing
  - replace back dummy nodes with bends





 copyright ©2025 m.patrignani

Aesthetics and compaction

	Minimum area	Minimum total edge length	Minimum maximum edge length
Graph 1			
Graph 2			
Graph 3			

 copyright ©2025 m.patrignani

Complexity of the compaction step

- Previous slide shows that it is not always possible to minimize the size of the drawing with respect to all three aesthetics
  - area
  - average edge length
    - or sum of edge lengths
  - maximum edge length
- Minimizing with respect to any of such aesthetics is NP-complete
  - [Patrignani 01]

 copyright ©2025 m.patrignani

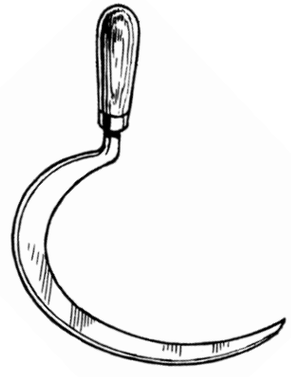
Compaction via rectangularization

- A two-step compaction algorithm
  1. Rectangularization
     - transform the orthogonal representation in such a way that each face is a rectangle
     - adds dummy nodes and dummy edges to the drawing
  2. Coordinate assignment
     - assign coherent and non-null lengths to the segments
     - produce the final x and y coordinates for vertices and bends

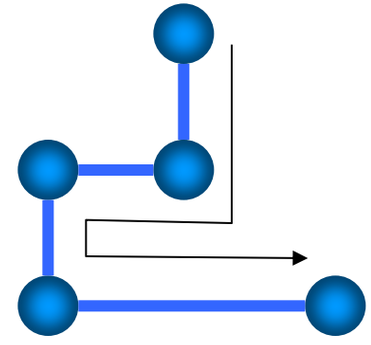
 copyright ©2025 m.patrignani

Rectangularization

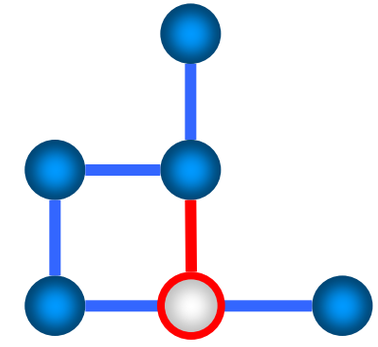
- Recursively search the boundary of a face for a sickle-shaped path
- Add a dummy node and a dummy vertex and split the face
  - can be done by adding O(n+b) dummy vertices and edges



sickle



sickle-shaped path

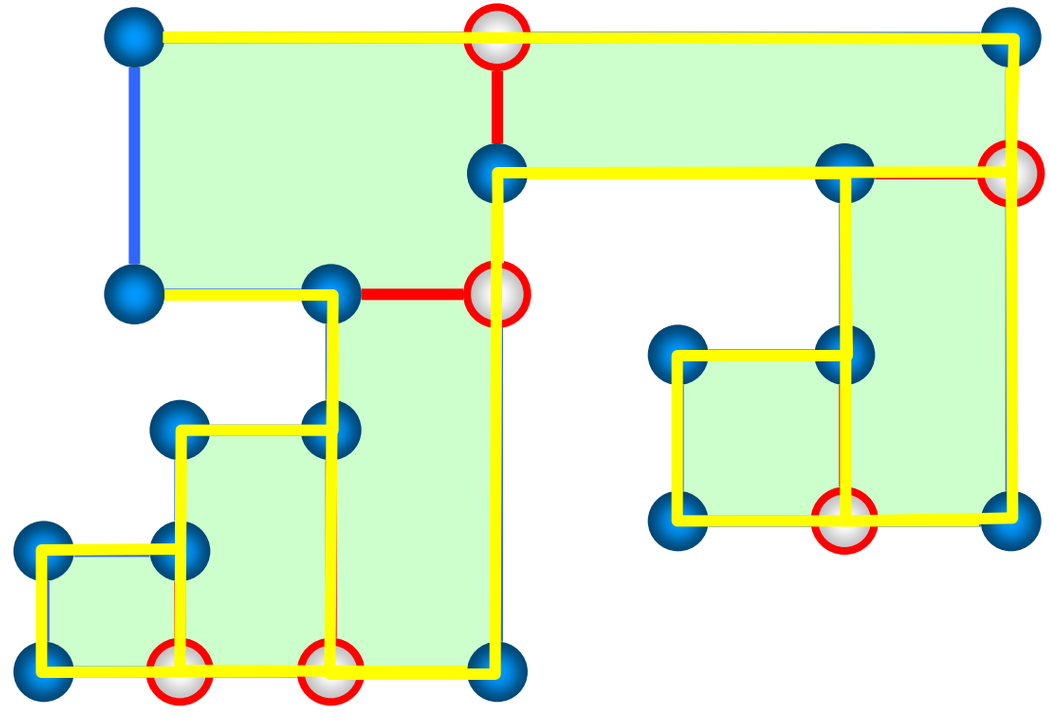


face split

copyright ©2025 m.patrignani

Compaction via rectangularization

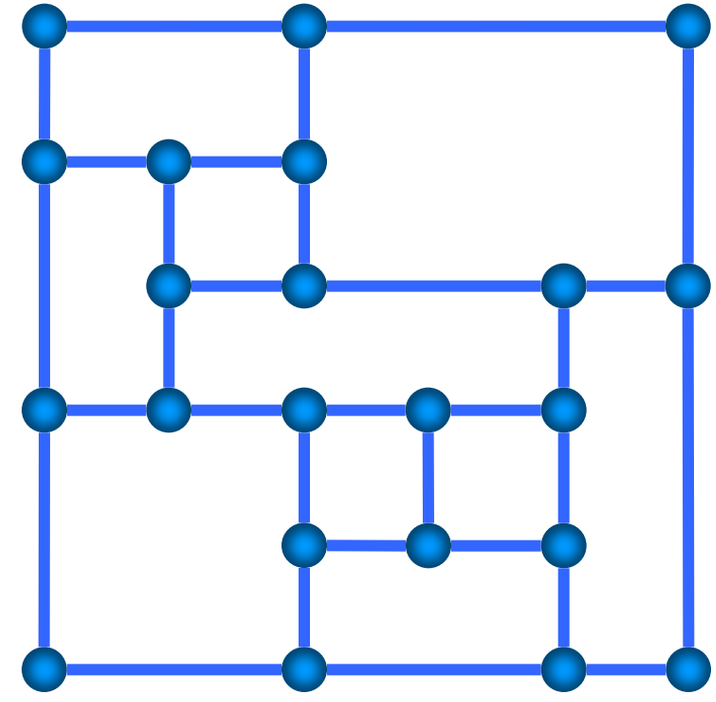
- Consider any internal face f
  - the whole graph can be enclosed into a rectangle in such a way that the external face is always rectangular



 copyright ©2025 m.patrignani

Lengths assignment

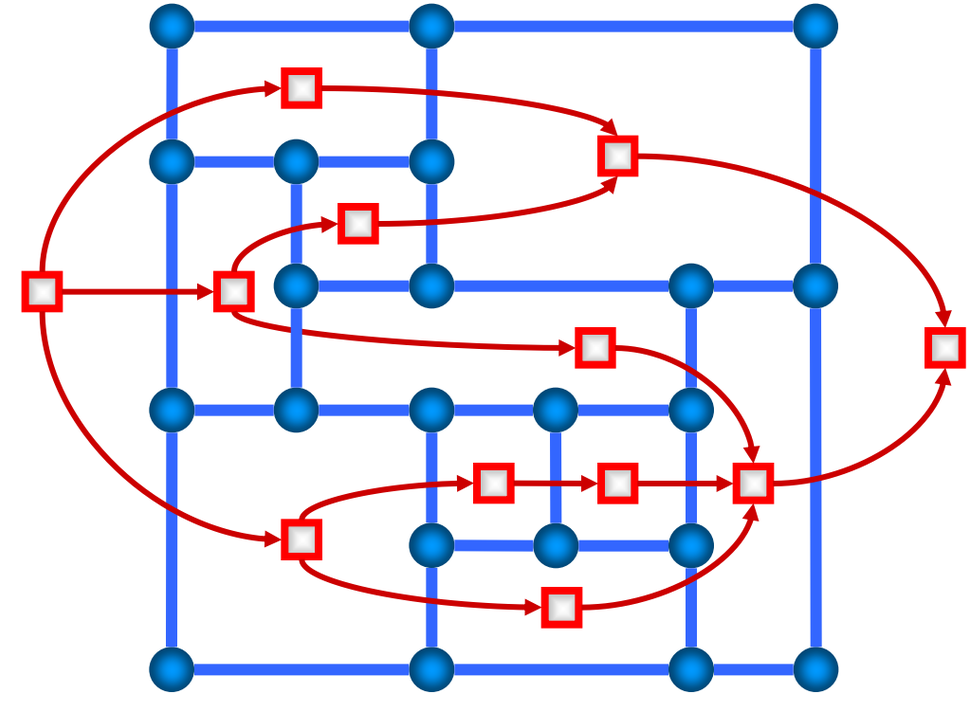
- All faces are rectangles
- In order to compute edge lengths we define two independent flow networks
  - one computes the lengths of vertical edges
  - the second computes the lengths of the horizontal edges



 copyright ©2025 m.patrignani

Vertical edge lengths

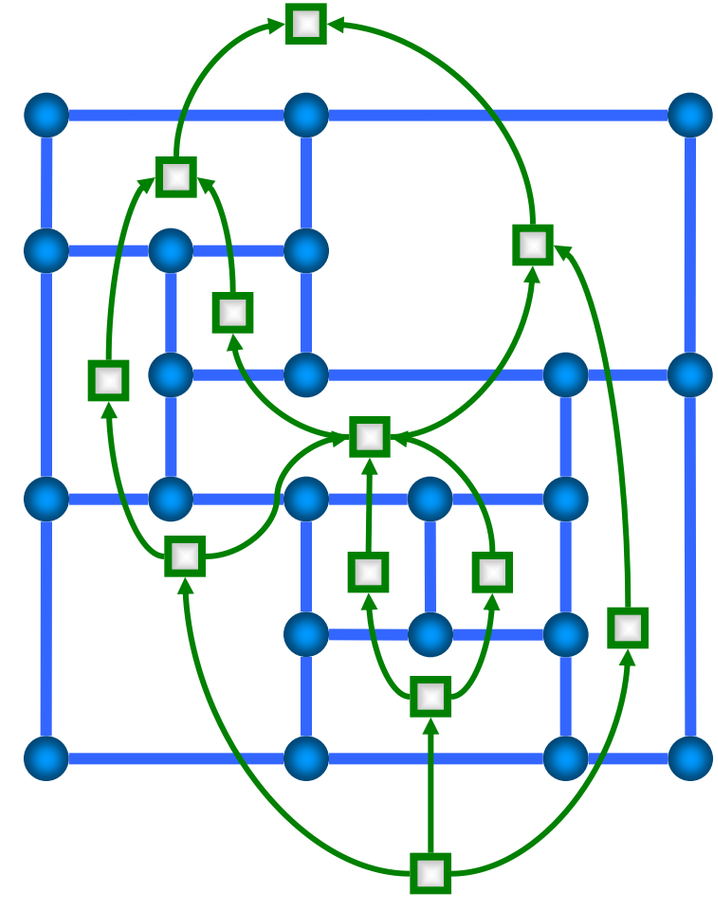
- One unit of flow represents the minimum distance among vertices
  - all costs are equal to 1
  - all lower bounds are equal to 1



 copyright ©2025 m.patrignani

Horizontal edge lengths

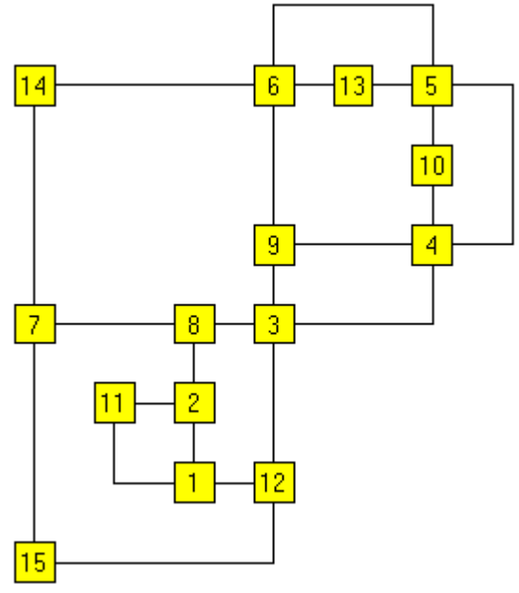
- The sum of edge lengths is minimized



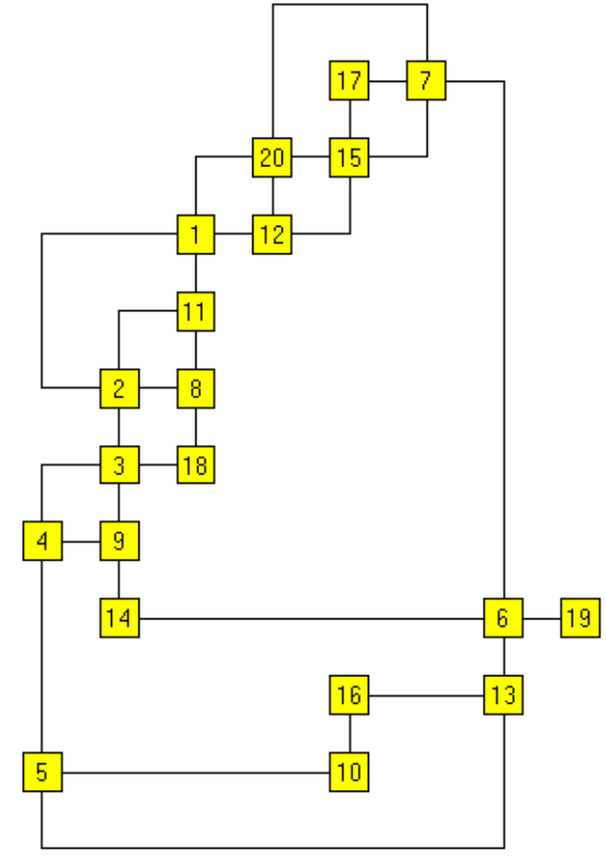
 copyright ©2025 m.patrignani

Examples of orthogonal drawings

- Drawings obtained with GDToolkit



15 nodes

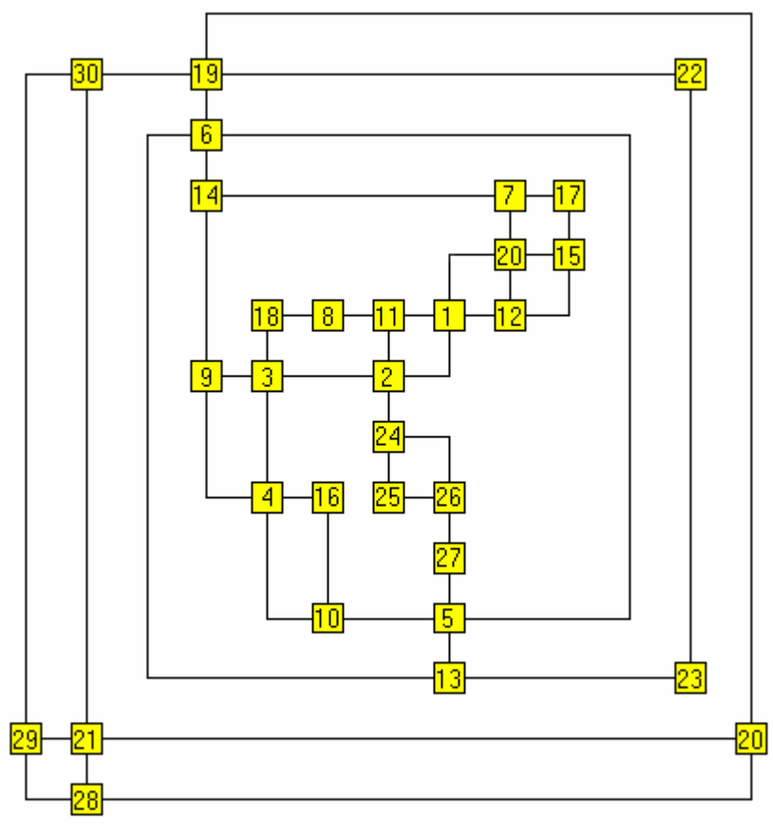


20 nodes

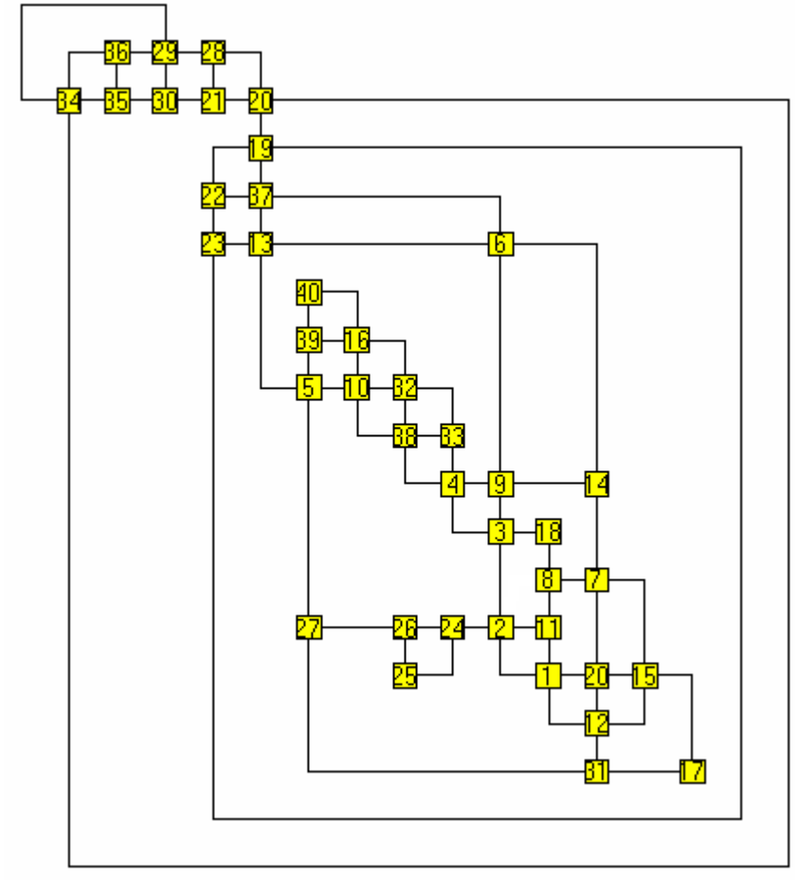
 copyright ©2025 m.patrignani

Examples of orthogonal drawings

- Drawings obtained with GDToolkit



30 nodes

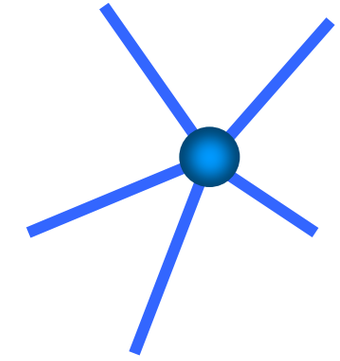


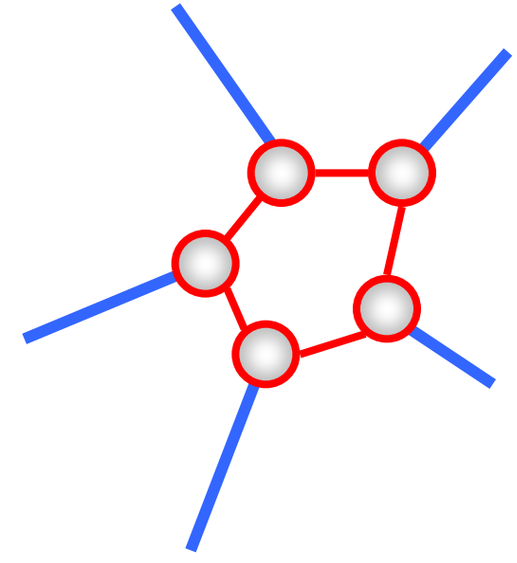
40 nodes

 copyright ©2025 m.patrignani

Handling general graphs

- Extension of the flow technique to graphs with nodes of arbitrary degree
- Each node v of degree deg(v) > 4 is replaced by a cycle $v_1, \ldots, v_d$
  - each v_i is incident to one edge of v
  - since the input graph is planar the replacement introduces a linear number of vertices and edges

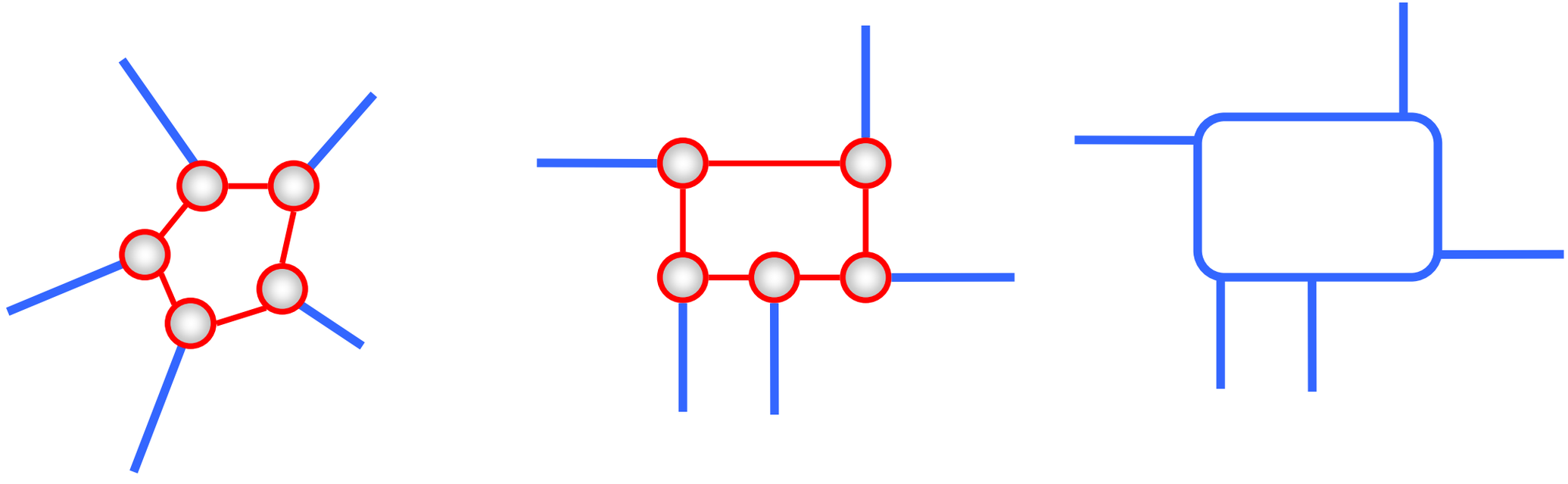




copyright ©2025 m.patrignani

Handling general graphs

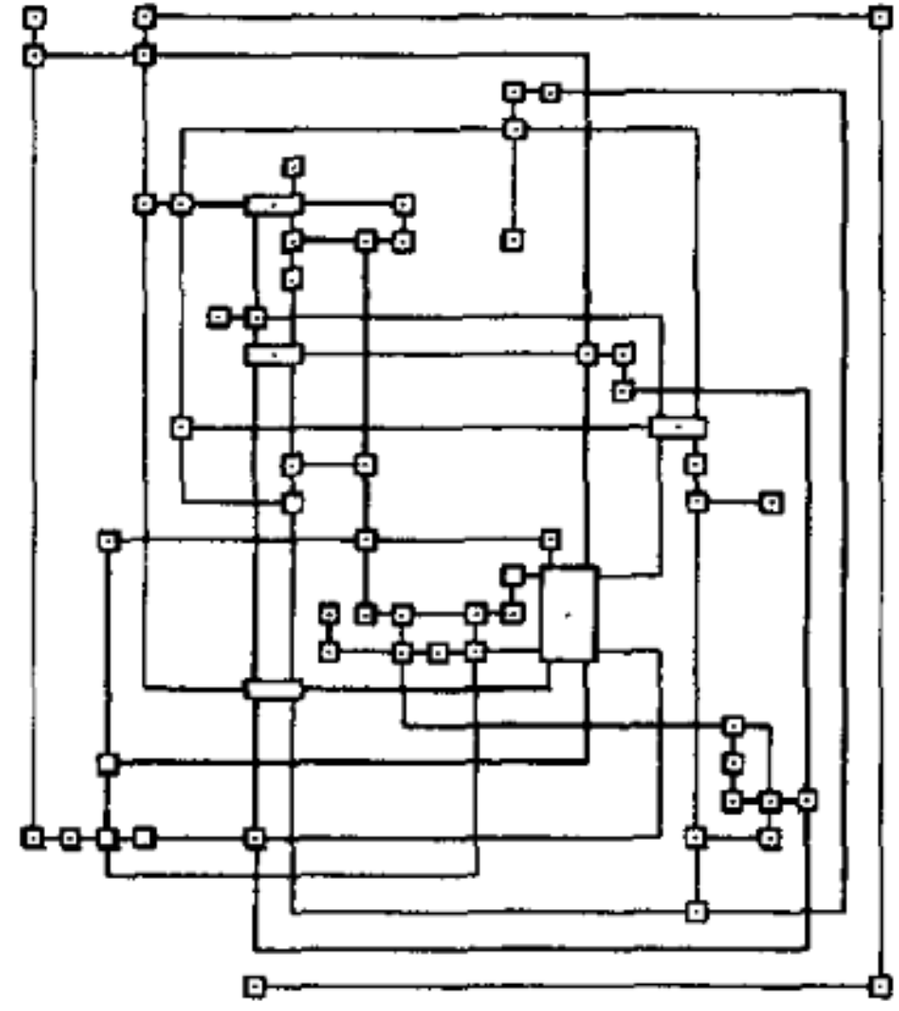
- Solve using a flow network N(G) where cycle edges are not allowed to bend
  - since edges of the cycle do not bend, the cycle can only turn inwards at some node
- Replace the cycle with a box



 copyright ©2025 m.patrignani

An example

- An orthogonal drawing of a general (non planar) graph obtained with Giotto
  - [Di Battista et al., 97]



 copyright ©2025 m.patrignani

Bibliography and further readings

- [Cornelsen & Karrenbauer, 12] S. Cornelsen and A. Karrenbauer, “Accelerated bend minimization”, J Graph Algorithms Appl, 2012
- [Di Battista et al., 97] G. Di Battista, A. Garg, G. Liotta, R. Tamassia, E. Tassinari, F. Vargiu, “An experimental comparison of four graph drawing algorithms”. Comput. Geom., 1997
  [Di Battista et al., 99] G. Di Battista, P. Eades, R. Tamassia, and I. G. Tollis, “Graph Drawing: Algorithms for the Visualization of Graphs”, Prentice-Hall, 1999
- [Fößmeier & Kaufmann , 96] U. Fößmeier and M. Kaufmann, “Drawing High Degree Graphs with Low Bend Numbers”, Graph Drawing, 1996
- [Garg & Tamassia, 96] A. Garg and R. Tamassia, “A new minimum cost flow algorithm with applications to graph drawing”, Graph Drawing, 1996

 copyright ©2025 m.patrignani

Bibliography and further readings

- [Garey & Johnson, 83] M. R. Garey and D. S. Johnson, “Crossing number is NP-complete”, SIAM J. Algebraic Discrete Methods, 1983
- [Hopcroft & Tarjan 74] J. Hopcroft and R. E. Tarjan. Efficient planarity testing. J. ACM, 1974
- [Mehlhorn & Mutzel 96] K. Mehlhorn and P. Mutzel. On the embedding phase of the Hopcroft and Tarjan planarity testing algorithm. Algorithmica, 1996
- [Patrignani 01] Maurizio Patrignani “On the Complexity of Orthogonal Compaction”, Computational Geometry: Theory and Applications, 2001
- [Storer 80] J.A. Storer, “The node cost measure for embedding graphs in the planar grid”, Proc. 12th ACM Symposium on the Theory of Computing, 1980
- [Tamassia 87] Roberto Tamassia “On Embedding a Graph in the Grid with the Minimum Number of Bends”, SIAM J. Computing, 16, 1987

 copyright ©2025 m.patrignani